



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

LAB MANUAL

Name: V.Madhulika

Register Name: 192312620

Course Code: ITA 0603

Course Name: Machine Learning

EXPERIMENT 1 – FIND-S ALGORITHM

Aim

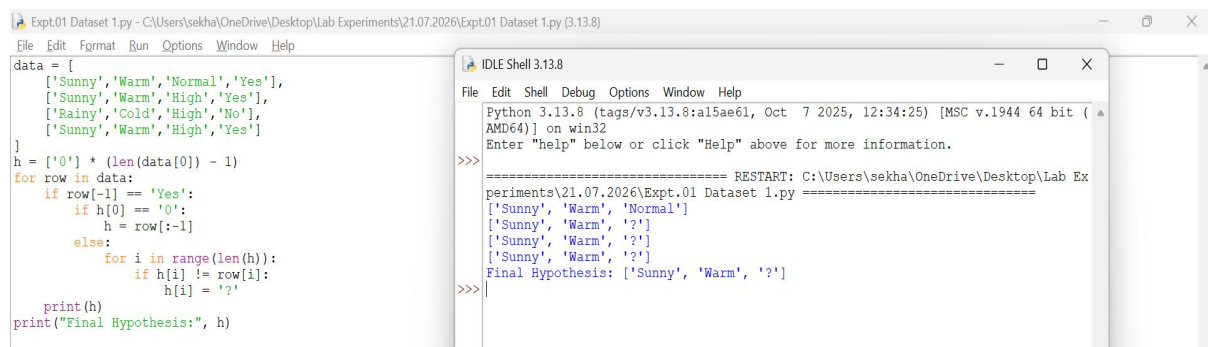
To implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.

Procedure

1. Define the training dataset.
2. Initialize the hypothesis with the most specific values.
3. Consider only positive training examples.
4. Compare positive examples with the hypothesis.
5. Replace differing attributes with '?'.
6. Ignore negative examples.
7. Display the final hypothesis.

Code and Output

1. Dataset 1

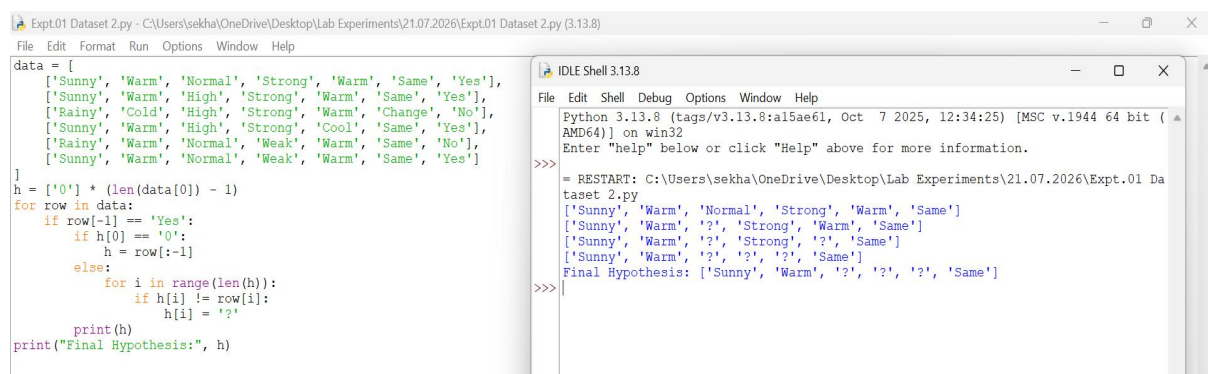


```
data = [
    ['Sunny', 'Warm', 'Normal', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Yes'],
    ['Rainy', 'Cold', 'High', 'No'],
    ['Sunny', 'Warm', 'High', 'Yes']
]
h = ['0'] * (len(data[0]) - 1)
for row in data:
    if row[-1] == 'Yes':
        if h[0] == '0':
            h = row[:-1]
        else:
            for i in range(len(h)):
                if h[i] != row[i]:
                    h[i] = '?'
    print(h)
print("Final Hypothesis:", h)
```

```
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

===== RESTART: C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt.01 Dataset 1.py =====
['Sunny', 'Warm', 'Normal']
['Sunny', 'Warm', '?']
['Sunny', 'Warm', '?']
['Sunny', 'Warm', '?']
Final Hypothesis: ['Sunny', 'Warm', '?']
```

2. Dataset 2



```
data = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Same', 'Yes'],
    ['Rainy', 'Warm', 'Normal', 'Weak', 'Warm', 'Same', 'No'],
    ['Sunny', 'Warm', 'Normal', 'Weak', 'Warm', 'Same', 'Yes']
]
h = ['0'] * (len(data[0]) - 1)
for row in data:
    if row[-1] == 'Yes':
        if h[0] == '0':
            h = row[:-1]
        else:
            for i in range(len(h)):
                if h[i] != row[i]:
                    h[i] = '?'
    print(h)
print("Final Hypothesis:", h)
```

```
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

===== RESTART: C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt.01 Dataset 2.py =====
['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
['Sunny', 'Warm', '?', 'Strong', '?', 'Same']
['Sunny', 'Warm', '?', '?', 'Same']
Final Hypothesis: ['Sunny', 'Warm', '?', '?', '?', 'Same']
```

3. Dataset 3

```
Expt01 Dataset 3.py - C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt01 Dataset 3.py (3.13.8)
File Edit Format Run Options Window Help

data = [
    ['High', 'Good', 'Yes', 'Good', 'High', 'Yes'],
    ['High', 'Excellent', 'Yes', 'Good', 'High', 'Yes'],
    ['Medium', 'Average', 'No', 'Average', 'Medium', 'No'],
    ['High', 'Good', 'Yes', 'Excellent', 'High', 'Yes'],
    ['Low', 'Poor', 'No', 'Average', 'Low', 'No'],
    ['High', 'Good', 'Yes', 'Good', 'Medium', 'Yes']
]
h = ['0'] * (len(data[0]) - 1)
for row in data:
    if row[-1] == 'Yes':
        if h[0] == '0':
            h = row[:-1]
        else:
            for i in range(len(h)):
                if h[i] != row[i]:
                    h[i] = '?'
    print(h)
print("Final Hypothesis:", h)

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt.01 Dataset 3.py
[['High', 'Good', 'Yes', 'Good', 'High', 'Yes'],
 ['High', '?', 'Yes', 'Good', 'High'],
 ['High', '?', 'Yes', '?', 'High'],
 ['High', '?', 'Yes', '?', 'High'],
 ['High', '?', 'Yes', '?', 'High'],
 ['High', '?', 'Yes', '?', 'High']]
Final Hypothesis: ['High', '?', 'Yes', '?', 'High']
>>>
```

4. Dataset 4

```
Expt01 Dataset 4.py - C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt01 Dataset 4.py (3.13.8)
File Edit Format Run Options Window Help

data = [
    ['High', 'Good', 'Yes', 'Good', 'High', 'Yes'],
    ['High', 'Excellent', 'Yes', 'Good', 'High', 'Yes'],
    ['Medium', 'Average', 'No', 'Average', 'Medium', 'No'],
    ['High', 'Good', 'Yes', 'Excellent', 'High', 'Yes'],
    ['Low', 'Poor', 'No', 'Average', 'Low', 'No'],
    ['High', 'Good', 'Yes', 'Good', 'Medium', 'Yes']
]
h = ['0'] * (len(data[0]) - 1)
for row in data:
    if row[-1] == 'Yes':
        if h[0] == '0':
            h = row[:-1]
        else:
            for i in range(len(h)):
                if h[i] != row[i]:
                    h[i] = '?'
    print(h)
print("Final Hypothesis:", h)

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt.01 Dataset 4.py
[['High', 'Good', 'Yes', 'Good', 'High', 'Yes'],
 ['High', '?', 'Yes', 'Good', 'High'],
 ['High', '?', 'Yes', '?', 'High'],
 ['High', '?', 'Yes', '?', 'High'],
 ['High', '?', 'Yes', '?', 'High'],
 ['High', '?', 'Yes', '?', 'High']]
Final Hypothesis: ['High', '?', 'Yes', '?', 'High']
>>>
```

5. Dataset 5

```
Expt01 Dataset 5.py - C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt01 Dataset 5.py (3.13.8)
File Edit Format Run Options Window Help

data = [
    ['Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'Positive'],
    ['Yes', 'Yes', 'No', 'Yes', 'Yes', 'Positive'],
    ['No', 'Yes', 'Yes', 'No', 'No', 'Negative'],
    ['Yes', 'Yes', 'Yes', 'No', 'Yes', 'Positive'],
    ['No', 'No', 'Yes', 'Yes', 'No', 'Negative'],
    ['Yes', 'Yes', 'No', 'No', 'Yes', 'Positive']
]
h = ['0'] * (len(data[0]) - 1)
for row in data:
    if row[-1] == 'Positive':
        if h[0] == '0':
            h = row[:-1]
        else:
            for i in range(len(h)):
                if h[i] != row[i]:
                    h[i] = '?'
    print(h)
print("Final Hypothesis:", h)

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt.01 Dataset 5.py
[['Yes', 'Yes', 'Yes', 'Yes', 'Yes'],
 ['Yes', 'Yes', '?', 'Yes', 'Yes'],
 ['Yes', 'Yes', '?', '?', 'Yes'],
 ['Yes', 'Yes', '?', '?', 'Yes'],
 ['Yes', 'Yes', '?', '?', 'Yes'],
 ['Yes', 'Yes', '?', '?', 'Yes']]
Final Hypothesis: ['Yes', 'Yes', '?', '?', 'Yes']
>>>
```

6. Dataset 6

```
Expt01 Dataset 6.py - C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt01 Dataset 6.py (3.13.8)
File Edit Format Run Options Window Help

data = [
    ['Yes', 'Yes', 'Yes', 'No', 'Yes', 'Yes'],
    ['Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes'],
    ['No', 'No', 'No', 'No', 'No', 'No'],
    ['Yes', 'No', 'Yes', 'No', 'Yes', 'No'],
    ['No', 'Yes', 'No', 'Yes', 'No', 'No'],
    ['Yes', 'Yes', 'Yes', 'No', 'No', 'Yes']
]
h = ['0'] * (len(data[0]) - 1)
for row in data:
    if row[-1] == 'Yes':
        if h[0] == '0':
            h = row[:-1]
        else:
            for i in range(len(h)):
                if h[i] != row[i]:
                    h[i] = '?'
    print(h)
print("Final Hypothesis:", h)

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt.01 Dataset 6.py
[['Yes', 'Yes', 'Yes', 'No', 'Yes'],
 ['Yes', 'Yes', 'Yes', '?', 'Yes'],
 ['Yes', '?', 'Yes', '?', 'Yes'],
 ['Yes', '?', 'Yes', '?', 'Yes'],
 ['Yes', '?', 'Yes', '?', 'Yes'],
 ['Yes', '?', 'Yes', '?', 'Yes']]
Final Hypothesis: ['Yes', '?', 'Yes', '?', 'Yes']
>>>
```

Result

Thus, the FIND-S algorithm was implemented successfully and the most specific hypothesis was obtained.

EXPERIMENT 2 – CANDIDATE ELIMINATION ALGORITHM

Aim

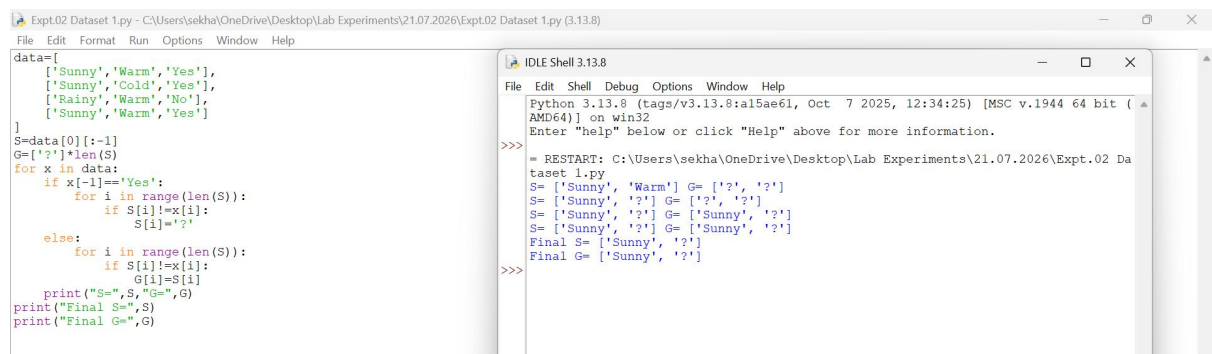
To implement and demonstrate the Candidate Elimination algorithm and obtain all hypotheses consistent with the training examples.

Procedure

1. Define the training dataset.
2. Initialize the Specific hypothesis (S).
3. Initialize the General hypothesis (G).
4. For positive examples, generalize S.
5. For negative examples, specialize G.
6. Update S and G after each iteration.
7. Display the final hypotheses.

Code and Output

1. Dataset 1



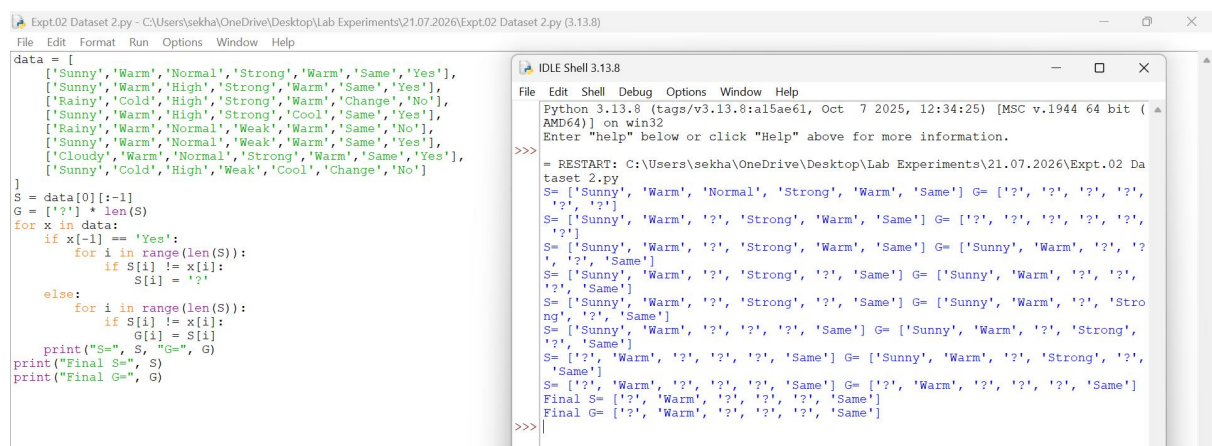
```
Expt.02 Dataset 1.py - C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt.02 Dataset 1.py (3.13.8)
File Edit Format Run Options Window Help

data=[
['Sunny','Warm','Yes'],
['Sunny','Cold','Yes'],
['Rainy','Warm','No'],
['Sunny','Warm','Yes']
]
S=data[0][:1]
G=['?']*len(S)
for x in data:
    if x[-1]=='Yes':
        for i in range(len(S)):
            if S[i]!=x[i]:
                S[i]='?'
    else:
        for i in range(len(S)):
            if S[i]!=x[i]:
                G[i]=S[i]
        print("S=",S,"G=",G)
print("Final S=",S)
print("Final G=",G)
```

```
IDLE Shell 3.13.8
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
= RESTART: C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt.02 Dataset 1.py
S= ['Sunny', 'Warm'] G= ['?', '?']
S= ['Sunny', '?'] G= ['?', '?']
S= ['Sunny', '?'] G= ['Sunny', '?']
S= ['Sunny', '?'] G= ['Sunny', '?']
Final S= ['Sunny', '?']
Final G= ['Sunny', '?']
>>>
```

2. Dataset 2



```
Expt.02 Dataset 2.py - C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt.02 Dataset 2.py (3.13.8)
File Edit Format Run Options Window Help

data = [
['Sunny','Warm','Normal','Strong','Warm','Same','Yes'],
['Sunny','Warm','High','Strong','Warm','Same','Yes'],
['Rainy','Cold','High','Strong','Warm','Change','No'],
['Sunny','Warm','High','Strong','Cool','Same','Yes'],
['Rainy','Warm','Normal','Weak','Warm','Same','No'],
['Sunny','Warm','Normal','Weak','Warm','Same','Yes'],
['Cloudy','Warm','Normal','Strong','Warm','Same','Yes'],
['Sunny','Cold','High','Weak','Cool','Change','No']
]
S = data[0][:1]
G = ['?'] * len(S)
for x in data:
    if x[-1] == 'Yes':
        for i in range(len(S)):
            if S[i] != x[i]:
                S[i] = '?'
    else:
        for i in range(len(S)):
            if S[i] != x[i]:
                G[i] = S[i]
        print("S=", S, "G=", G)
print("Final S=", S)
print("Final G=", G)
```

```
IDLE Shell 3.13.8
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
= RESTART: C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt.02 Dataset 2.py
S= ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same'] G= ['?', '?', '?', '?', '?', '?']
S= ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same'] G= ['?', '?', '?', '?', '?', '?']
S= ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same'] G= ['Sunny', 'Warm', '?', '?', '?', '?']
S= ['Sunny', 'Warm', '?', 'Strong', '?', 'Same'] G= ['Sunny', 'Warm', '?', 'Strong', '?', 'Same']
S= ['Sunny', 'Warm', '?', 'Strong', '?', 'Same'] G= ['Sunny', 'Warm', '?', 'Strong', '?', 'Same']
S= ['?', 'Warm', '?', '?', '?', 'Same'] G= ['Sunny', 'Warm', '?', 'Strong', '?', 'Same']
Final S= ['?', 'Warm', '?', '?', '?', 'Same'] G= ['?', 'Warm', '?', '?', '?', 'Same']
Final G= ['?', 'Warm', '?', '?', '?', 'Same']
>>>
```

3. Dataset 3


```
Expt.02 Dataset 3.py - C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt.02 Dataset 3.py (3.13.8)
File Edit Format Run Options Window Help

data = [
    ['High','Good','Permanent','Yes','Young','Yes'],
    ['High','Good','Permanent','No','Middle','Yes'],
    ['Low','Poor','Temporary','No','Young','No'],
    ['Medium','Good','Permanent','Yes','Middle','Yes'],
    ['High','Average','Temporary','Yes','Old','No'],
    ['Medium','Good','Permanent','No','Young','Yes'],
    ['Low','Good','Permanent','Yes','Middle','Yes'],
    ['Low','Poor','Temporary','Yes','Old','No']
]
S = data[0][:-1]
G = ['?'] * len(S)
for x in data:
    if x[-1] == 'Yes':
        for i in range(len(S)):
            if S[i] != x[i]:
                S[i] = '?'
    else:
        for i in range(len(S)):
            if S[i] != x[i]:
                G[i] = S[i]
        print("S=", S, "G=", G)
print("Final S=", S)
print("Final G=", G)

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> = RESTART: C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt.02 Dataset 3.py
S= ['High', 'Good', 'Permanent', 'Yes', 'Young'] G= ['?', '?', '?', '?', '?']
S= ['High', 'Good', 'Permanent', '?', '?'] G= ['?', '?', '?', '?', '?']
S= ['High', 'Good', 'Permanent', '?', '?'] G= ['High', 'Good', 'Permanent', '?', '?']
S= ['?', 'Good', 'Permanent', '?', '?'] G= ['High', 'Good', 'Permanent', '?', '?']
S= ['?', 'Good', 'Permanent', '?', '?'] G= ['High', 'Good', 'Permanent', '?', '?']
S= ['?', 'Good', 'Permanent', '?', '?'] G= ['?', 'Good', 'Permanent', '?', '?']
S= ['?', 'Good', 'Permanent', '?', '?'] G= ['?', 'Good', 'Permanent', '?', '?']
S= ['?', 'Good', 'Permanent', '?', '?'] G= ['?', 'Good', 'Permanent', '?', '?']
Final S= ['?', 'Good', 'Permanent', '?', '?']
Final G= ['?', 'Good', 'Permanent', '?', '?']
>>>
```

4. Dataset 4

```
Expt.02 Dataset 4.py - C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt.02 Dataset 4.py (3.13.8)
File Edit Format Run Options Window Help

data = [
    ['High','Good','Yes','Good','High','Yes'],
    ['High','Excellent','Yes','Good','High','Yes'],
    ['Medium','Average','No','Average','Medium','No'],
    ['High','Good','Yes','Excellent','High','Yes'],
    ['Low','Poor','No','Average','Low','No'],
    ['High','Good','Yes','Good','Medium','Yes'],
    ['Medium','Good','Yes','Good','High','Yes'],
    ['Low','Average','No','Poor','Medium','No']
]
S = data[0][:-1]
G = ['?'] * len(S)
for x in data:
    if x[-1] == 'Yes':
        for i in range(len(S)):
            if S[i] != x[i]:
                S[i] = '?'
    else:
        for i in range(len(S)):
            if S[i] != x[i]:
                G[i] = S[i]
        print("S=", S, "G=", G)
print("Final S=", S)
print("Final G=", G)

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> = RESTART: C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt.02 Dataset 4.py
S= ['High', 'Good', 'Yes', 'Good', 'High'] G= ['?', '?', '?', '?', '?']
S= ['High', '?', 'Yes', 'Good', 'High'] G= ['?', '?', '?', '?', '?']
S= ['High', '?', 'Yes', 'Good', 'High'] G= ['High', '?', 'Yes', 'Good', 'High']
S= ['High', '?', 'Yes', '?', 'High'] G= ['High', '?', 'Yes', '?', 'High']
S= ['High', '?', 'Yes', '?', 'High'] G= ['High', '?', 'Yes', '?', 'High']
S= ['?', '?', 'Yes', '?', 'High'] G= ['High', '?', 'Yes', '?', 'High']
S= ['?', '?', 'Yes', '?', 'High'] G= ['?', '?', 'Yes', '?', 'High']
S= ['?', '?', 'Yes', '?', 'High'] G= ['?', '?', 'Yes', '?', 'High']
Final S= ['?', '?', 'Yes', '?', 'High']
Final G= ['?', '?', 'Yes', '?', 'High']
>>>
```

5. Dataset 5

```
Expt.02 Dataset 5.py - C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt.02 Dataset 5.py (3.13.8)
File Edit Format Run Options Window Help

data = [
    ['Yes','Yes','Yes','Yes','No','Yes','Positive'],
    ['Yes','Yes','No','Yes','Yes','Positive'],
    ['No','Yes','Yes','No','No','Negative'],
    ['Yes','Yes','Yes','No','Yes','Positive'],
    ['No','No','Yes','Yes','No','Negative'],
    ['Yes','Yes','No','No','Yes','Positive'],
    ['Yes','No','Yes','Yes','Yes','Positive'],
    ['No','No','No','No','No','Negative']
]
S = data[0][:-1]
G = ['?'] * len(S)
for x in data:
    if x[-1] == 'Positive':
        for i in range(len(S)):
            if S[i] != x[i]:
                S[i] = '?'
    else:
        for i in range(len(S)):
            if S[i] != x[i]:
                G[i] = S[i]
        print("S=", S, "G=", G)
print("Final S=", S)
print("Final G=", G)

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> = RESTART: C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt.02 Dataset 5.py
S= ['Yes', 'Yes', 'Yes', 'Yes', 'Yes'] G= ['?', '?', '?', '?', '?']
S= ['Yes', 'Yes', '?', 'Yes', 'Yes'] G= ['Yes', '?', '?', 'Yes', 'Yes']
S= ['Yes', 'Yes', '?', '?', 'Yes'] G= ['Yes', '?', '?', 'Yes', 'Yes']
S= ['Yes', 'Yes', '?', '?', 'Yes'] G= ['Yes', 'Yes', '?', '?', 'Yes']
S= ['Yes', 'Yes', '?', '?', 'Yes'] G= ['Yes', 'Yes', '?', '?', 'Yes']
S= ['Yes', '?', '?', '?', 'Yes'] G= ['Yes', 'Yes', '?', '?', 'Yes']
S= ['Yes', '?', '?', '?', 'Yes'] G= ['Yes', '?', '?', '?', 'Yes']
Final S= ['Yes', '?', '?', '?', 'Yes']
Final G= ['Yes', '?', '?', '?', 'Yes']
>>>
```

6. Dataset 6

```
Expt.02 Dataset 6.py - C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt.02 Dataset 6.py (3.13.8)
File Edit Format Run Options Window Help

data = [
    ['Yes','Yes','Yes','Yes','No','Yes','Yes'],
    ['Yes','Yes','Yes','Yes','Yes','Yes'],
    ['No','No','No','No','No','No'],
    ['Yes','No','Yes','No','Yes','Yes'],
    ['No','Yes','No','Yes','No','No'],
    ['Yes','Yes','Yes','No','No','Yes'],
    ['Yes','No','Yes','Yes','Yes','Yes'],
    ['No','No','Yes','No','No','No']
]
S = data[0][:-1]
G = ['?'] * len(S)
for x in data:
    if x[-1] == 'Yes':
        for i in range(len(S)):
            if S[i] != x[i]:
                S[i] = '?'
    else:
        for i in range(len(S)):
            if S[i] != x[i]:
                G[i] = S[i]
        print("S=", S, "G=", G)
print("Final S=", S)
print("Final G=", G)

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> = RESTART: C:\Users\sekha\OneDrive\Desktop\Lab Experiments\21.07.2026\Expt.02 Dataset 6.py
S= ['Yes', 'Yes', 'Yes', 'Yes', 'Yes'] G= ['?', '?', '?', '?', '?']
S= ['Yes', 'Yes', 'Yes', '?', 'Yes'] G= ['?', '?', '?', '?', '?']
S= ['Yes', 'Yes', 'Yes', '?', 'Yes'] G= ['Yes', 'Yes', 'Yes', '?', 'Yes']
S= ['Yes', '?', 'Yes', '?', 'Yes'] G= ['Yes', 'Yes', 'Yes', '?', 'Yes']
S= ['Yes', '?', 'Yes', '?', 'Yes'] G= ['Yes', '?', 'Yes', '?', 'Yes']
S= ['Yes', '?', 'Yes', '?', 'Yes'] G= ['Yes', '?', 'Yes', '?', 'Yes']
S= ['Yes', '?', 'Yes', '?', 'Yes'] G= ['Yes', '?', 'Yes', '?', 'Yes']
S= ['Yes', '?', 'Yes', '?', 'Yes'] G= ['Yes', '?', 'Yes', '?', 'Yes']
Final S= ['Yes', '?', 'Yes', '?', 'Yes']
Final G= ['Yes', '?', 'Yes', '?', 'Yes']
>>>
```

Result

Thus, the Candidate Elimination algorithm was implemented successfully and the final Specific and General hypotheses were obtained.

EXPERIMENT 3 – DECISION TREE (ID3) ALGORITHM

Aim

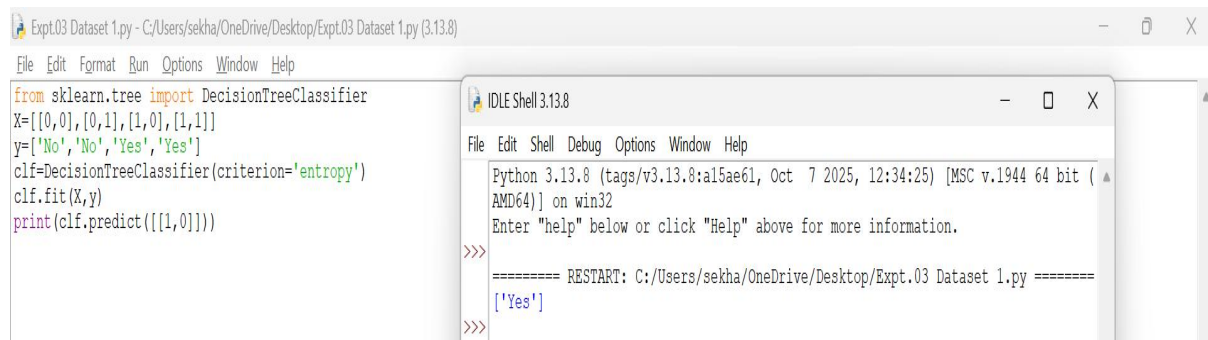
To implement and demonstrate the working of the Decision Tree based ID3 algorithm and classify a new sample.

Procedure

1. Import the Decision Tree Classifier.
2. Create the training dataset and target values.
3. Create the Decision Tree model using entropy.
4. Train the model using the training data.
5. Classify a new sample using the trained model.
6. Display the predicted result.

Code and Output

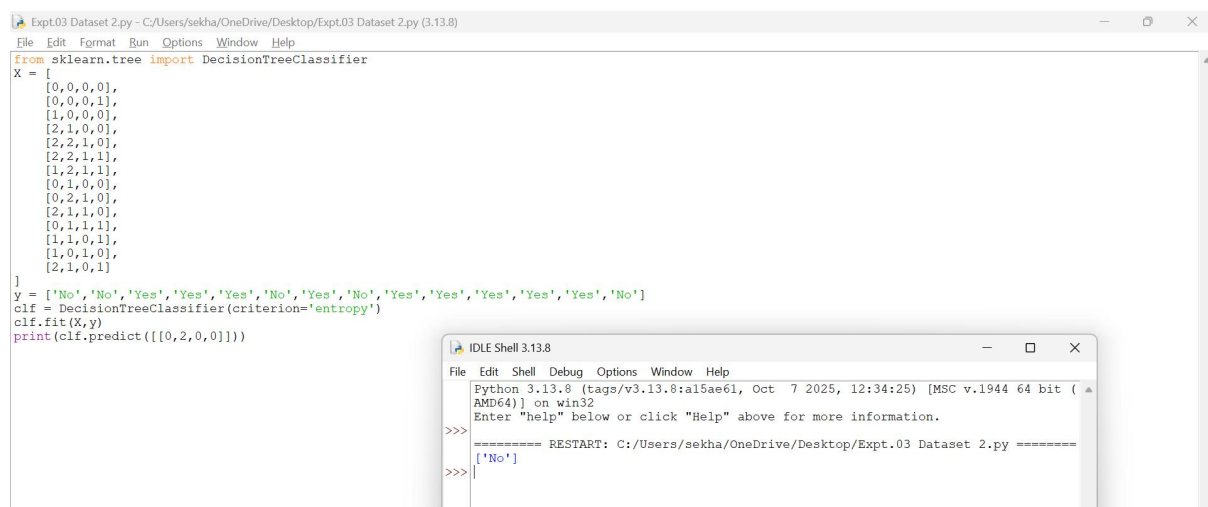
1. Dataset 1



```
Expt.03 Dataset 1.py - C:/Users/sekha/OneDrive/Desktop/Expt.03 Dataset 1.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.tree import DecisionTreeClassifier
X=[[0,0],[0,1],[1,0],[1,1]]
y=['No','No','Yes','Yes']
clf=DecisionTreeClassifier(criterion='entropy')
clf.fit(X,y)
print(clf.predict([[1,0]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/sekha/OneDrive/Desktop/Expt.03 Dataset 1.py =====
['Yes']
>>>
```

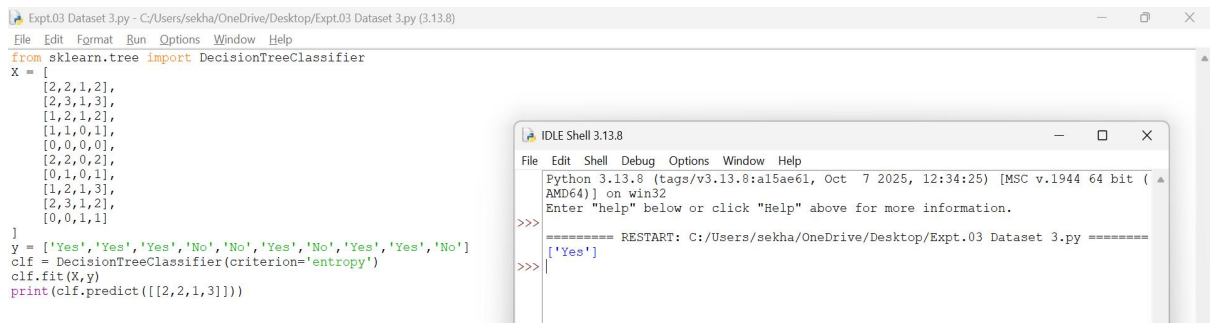
2. Dataset 2



```
Expt.03 Dataset 2.py - C:/Users/sekha/OneDrive/Desktop/Expt.03 Dataset 2.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.tree import DecisionTreeClassifier
X = [
[0,0,0,0],
[0,0,0,1],
[1,0,0,0],
[2,1,0,0],
[2,2,1,0],
[2,2,1,1],
[1,2,1,1],
[0,1,0,0],
[0,2,1,0],
[2,1,1,0],
[0,1,1,1],
[1,1,0,1],
[1,0,1,0],
[2,1,0,1]
]
y = ['No','No','Yes','Yes','Yes','No','Yes','No','Yes','Yes','Yes','Yes','Yes','No']
clf = DecisionTreeClassifier(criterion='entropy')
clf.fit(X,y)
print(clf.predict([[0,2,0,0]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/sekha/OneDrive/Desktop/Expt.03 Dataset 2.py =====
['No']
>>>
```

3. Dataset 3

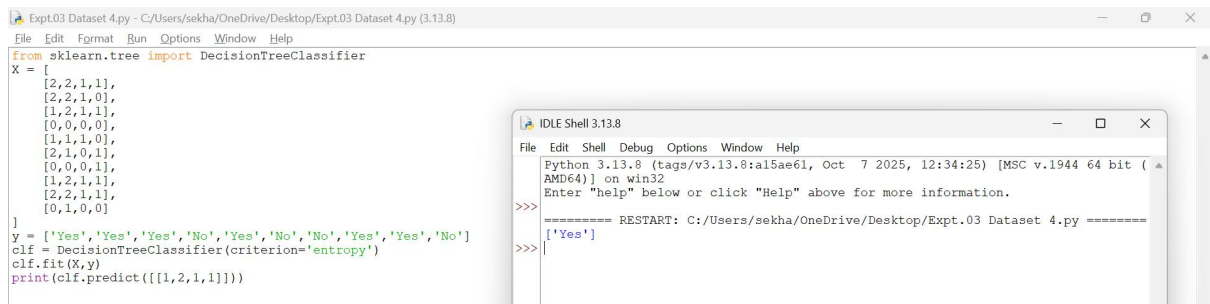


The screenshot shows a Python IDE window titled 'Expt.03 Dataset 3.py - C:/Users/sekha/OneDrive/Desktop/Expt.03 Dataset 3.py (3.13.8)'. The code imports `DecisionTreeClassifier` from `sklearn.tree`, defines a feature matrix `X` and a target vector `y`, fits a decision tree classifier, and prints the prediction for the sample `[2, 2, 1, 3]`. An 'IDLE Shell 3.13.8' window is open, showing the execution output: `['Yes']`.

```
from sklearn.tree import DecisionTreeClassifier
X = [
    [2,2,1,2],
    [2,3,1,3],
    [1,2,1,2],
    [1,1,0,1],
    [0,0,0,0],
    [2,2,0,2],
    [0,1,0,1],
    [1,2,1,3],
    [2,3,1,2],
    [0,0,1,1]
]
y = ['Yes','Yes','Yes','No','No','Yes','No','Yes','Yes','No']
clf = DecisionTreeClassifier(criterion='entropy')
clf.fit(X,y)
print(clf.predict([[2,2,1,3]]))
```

```
>>>
===== RESTART: C:/Users/sekha/OneDrive/Desktop/Expt.03 Dataset 3.py =====
>>> ['Yes']
```

4. Dataset 4

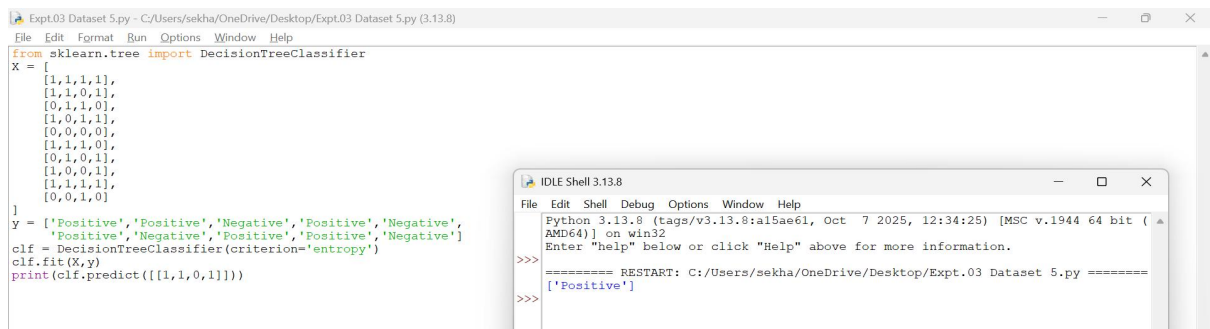


The screenshot shows a Python IDE window titled 'Expt.03 Dataset 4.py - C:/Users/sekha/OneDrive/Desktop/Expt.03 Dataset 4.py (3.13.8)'. The code imports `DecisionTreeClassifier` from `sklearn.tree`, defines a feature matrix `X` and a target vector `y`, fits a decision tree classifier, and prints the prediction for the sample `[1, 2, 1, 1]`. An 'IDLE Shell 3.13.8' window is open, showing the execution output: `['Yes']`.

```
from sklearn.tree import DecisionTreeClassifier
X = [
    [2,2,1,1],
    [2,2,1,0],
    [1,2,1,1],
    [0,0,0,0],
    [1,1,1,0],
    [2,1,0,1],
    [0,0,0,1],
    [1,2,1,1],
    [2,2,1,1],
    [0,1,0,0]
]
y = ['Yes','Yes','Yes','No','Yes','No','No','Yes','Yes','No']
clf = DecisionTreeClassifier(criterion='entropy')
clf.fit(X,y)
print(clf.predict([[1,2,1,1]]))
```

```
>>>
===== RESTART: C:/Users/sekha/OneDrive/Desktop/Expt.03 Dataset 4.py =====
>>> ['Yes']
```

5. Dataset 5

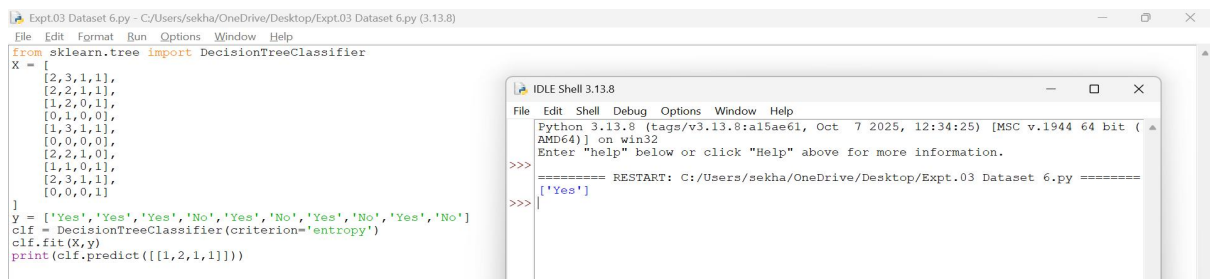


The screenshot shows a Python IDE window titled 'Expt.03 Dataset 5.py - C:/Users/sekha/OneDrive/Desktop/Expt.03 Dataset 5.py (3.13.8)'. The code imports `DecisionTreeClassifier` from `sklearn.tree`, defines a feature matrix `X` and a target vector `y`, fits a decision tree classifier, and prints the prediction for the sample `[1, 1, 0, 1]`. An 'IDLE Shell 3.13.8' window is open, showing the execution output: `['Positive']`.

```
from sklearn.tree import DecisionTreeClassifier
X = [
    [1,1,1,1],
    [1,1,0,1],
    [0,1,1,0],
    [1,0,1,1],
    [0,0,0,0],
    [1,1,1,0],
    [0,1,0,1],
    [1,0,0,1],
    [1,1,1,1],
    [0,0,1,0]
]
y = ['Positive','Positive','Negative','Positive','Negative',
     'Positive','Negative','Positive','Positive','Negative']
clf = DecisionTreeClassifier(criterion='entropy')
clf.fit(X,y)
print(clf.predict([[1,1,0,1]]))
```

```
>>>
===== RESTART: C:/Users/sekha/OneDrive/Desktop/Expt.03 Dataset 5.py =====
>>> ['Positive']
```

6. Dataset 6



The screenshot shows a Python IDE window titled 'Expt.03 Dataset 6.py - C:/Users/sekha/OneDrive/Desktop/Expt.03 Dataset 6.py (3.13.8)'. The code imports `DecisionTreeClassifier` from `sklearn.tree`, defines a feature matrix `X` and a target vector `y`, fits a decision tree classifier, and prints the prediction for the sample `[1, 2, 1, 1]`. An 'IDLE Shell 3.13.8' window is open, showing the execution output: `['Yes']`.

```
from sklearn.tree import DecisionTreeClassifier
X = [
    [2,3,1,1],
    [2,2,1,1],
    [1,2,0,1],
    [0,1,0,0],
    [1,3,1,1],
    [0,0,0,0],
    [2,2,1,0],
    [1,1,0,1],
    [2,3,1,1],
    [0,0,0,1]
]
y = ['Yes','Yes','Yes','No','Yes','No','Yes','No','Yes','No']
clf = DecisionTreeClassifier(criterion='entropy')
clf.fit(X,y)
print(clf.predict([[1,2,1,1]]))
```

```
>>>
===== RESTART: C:/Users/sekha/OneDrive/Desktop/Expt.03 Dataset 6.py =====
>>> ['Yes']
```

Result

Thus, the Decision Tree based ID3 algorithm was implemented successfully and the new sample was classified as 'Yes'.

EXPERIMENT 4 – ARTIFICIAL NEURAL NETWORK USING BACKPROPAGATION

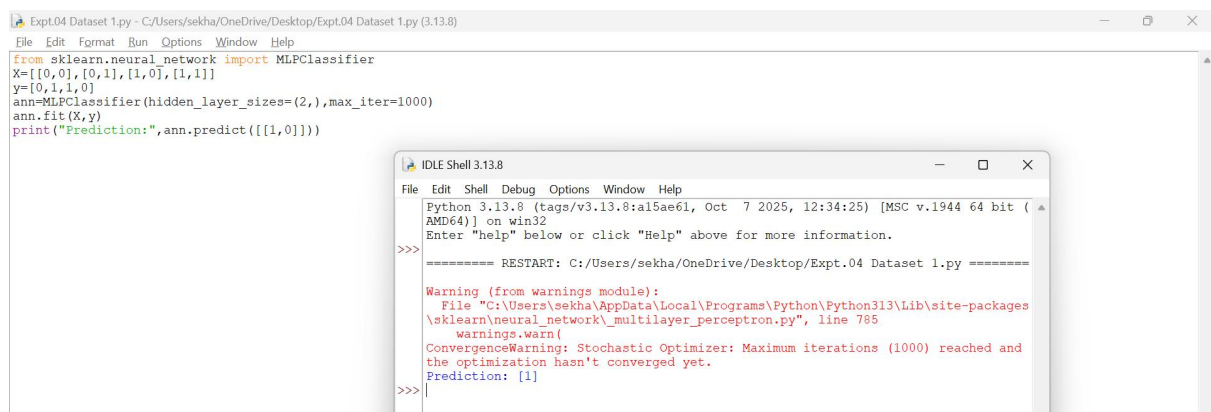
Aim

To build an Artificial Neural Network using the Backpropagation algorithm and classify a new sample.

Procedure

1. Import the MLPClassifier.
2. Create the training dataset and target values.
3. Create the ANN model.
4. Train the model using Backpropagation.
5. Test the model using a new sample.
6. Display the predicted result.

Code and Output



The screenshot shows a Python script in a text editor and its execution in an IDLE Shell. The script imports MLPClassifier from sklearn.neural_network, creates a training dataset X and target values y, trains the model, and predicts the result for a new sample.

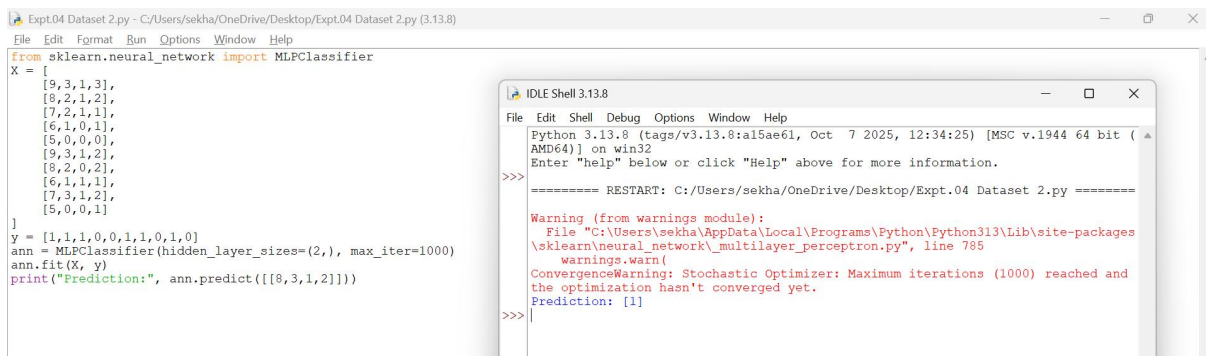
```
from sklearn.neural_network import MLPClassifier
X=[[0,0],[0,1],[1,0],[1,1]]
y=[0,1,1,0]
ann=MLPClassifier(hidden_layer_sizes=(2,),max_iter=1000)
ann.fit(X,y)
print("Prediction:",ann.predict([[1,0]]))
```

The IDLE Shell output shows a warning about the optimization not converging and the prediction result [1].

```
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/sekha/OneDrive/Desktop/Expt.04 Dataset 1.py =====
Warning (from warnings module):
  File "C:/Users/sekha/AppData/Local/Programs/Python/Python313/Lib/site-packages/sklearn/neural_network/multilayer_perceptron.py", line 785
    warnings.warn(
ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
Prediction: [1]
>>>
```

1. Dataset 1
2. Dataset 2



The screenshot shows a Python script in a text editor and its execution in an IDLE Shell. The script imports MLPClassifier from sklearn.neural_network, creates a training dataset X and target values y, trains the model, and predicts the result for a new sample.

```
from sklearn.neural_network import MLPClassifier
X = [
    [9,3,1,3],
    [8,2,1,2],
    [7,2,1,1],
    [6,1,0,1],
    [5,0,0,0],
    [9,3,1,2],
    [8,2,0,2],
    [6,1,1,1],
    [7,3,1,2],
    [5,0,0,1]
]
y = [1,1,1,0,0,1,1,0,1,0]
ann = MLPClassifier(hidden_layer_sizes=(2,), max_iter=1000)
ann.fit(X, y)
print("Prediction:", ann.predict([[8,3,1,2]]))
```

The IDLE Shell output shows a warning about the optimization not converging and the prediction result [1].

```
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/sekha/OneDrive/Desktop/Expt.04 Dataset 2.py =====
Warning (from warnings module):
  File "C:/Users/sekha/AppData/Local/Programs/Python/Python313/Lib/site-packages/sklearn/neural_network/multilayer_perceptron.py", line 785
    warnings.warn(
ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
Prediction: [1]
>>>
```

3. Dataset 3

The screenshot shows a Python IDE window titled 'Expt.04 Dataset 3.py - C:/Users/sekha/OneDrive/Desktop/Expt.04 Dataset 3.py (3.13.8)'. The code imports `MLPClassifier` from `sklearn.neural_network` and defines a dataset `X` with 10 samples and a target vector `y`. It then fits an `MLPClassifier` with `hidden_layer_sizes=(2,)` and `max_iter=1000`, and prints the prediction for the test sample `[1, 2, 1, 1]`. An adjacent 'IDLE Shell 3.13.8' window shows the execution output, including a `ConvergenceWarning` about the maximum iterations being reached and the prediction result: `Prediction: [1]`.

```
from sklearn.neural_network import MLPClassifier
X = [
    [2,2,1,1],
    [2,2,1,0],
    [1,2,1,1],
    [1,1,1,0],
    [0,0,0,0],
    [0,1,0,1],
    [2,1,1,1],
    [1,2,0,0],
    [2,2,1,1],
    [0,0,0,1]
]
y = [1,1,1,1,0,0,1,0,1,0]
ann = MLPClassifier(hidden_layer_sizes=(2,), max_iter=1000)
ann.fit(X, y)
print("Prediction:", ann.predict([[1,2,1,1]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1444 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

===== RESTART: C:/Users/sekha/OneDrive/Desktop/Expt.04 Dataset 3.py =====

Warning (from warnings module):
  File "C:/Users/sekha/AppData/Local/Programs/Python/Python313/Lib/site-packages/sklearn/neural_network/multilayer_perceptron.py", line 785
    warnings.warn(
ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
Prediction: [1]
```

4. Dataset 4

The screenshot shows a Python IDE window titled 'Expt.04 Dataset 4.py - C:/Users/sekha/OneDrive/Desktop/Expt.04 Dataset 4.py (3.13.8)'. The code defines a dataset `X` with 10 samples and a target vector `y`. It then fits an `MLPClassifier` with `hidden_layer_sizes=(2,)` and `max_iter=1000`, and prints the prediction for the test sample `[1, 1, 0, 1]`. The 'IDLE Shell 3.13.8' window shows the execution output, including a `ConvergenceWarning` and the prediction result: `Prediction: [1]`.

```
from sklearn.neural_network import MLPClassifier
X = [
    [1,1,1,1,1],
    [1,1,0,1,1],
    [0,1,1,0,0],
    [1,0,1,1,1],
    [0,0,0,0,0],
    [1,1,1,0,1],
    [0,1,0,1,0],
    [1,0,0,1,1],
    [1,1,1,1,1],
    [0,0,1,0,0]
]
y = [1,1,0,1,0,1,0,1,1,0]
ann = MLPClassifier(hidden_layer_sizes=(2,), max_iter=1000)
ann.fit(X, y)
print("Prediction:", ann.predict([[1,1,0,1]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1444 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

===== RESTART: C:/Users/sekha/OneDrive/Desktop/Expt.04 Dataset 4.py =====

Warning (from warnings module):
  File "C:/Users/sekha/AppData/Local/Programs/Python/Python313/Lib/site-packages/sklearn/neural_network/multilayer_perceptron.py", line 785
    warnings.warn(
ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
Prediction: [1]
```

5. Dataset 5

The screenshot shows a Python IDE window titled 'Expt.04 Dataset 5.py - C:/Users/sekha/OneDrive/Desktop/Expt.04 Dataset 5.py (3.13.8)'. The code defines a dataset `X` with 10 samples and a target vector `y`. It then fits an `MLPClassifier` with `hidden_layer_sizes=(2,)` and `max_iter=1000`, and prints the prediction for the test sample `[1, 2, 1, 1]`. The 'IDLE Shell 3.13.8' window shows the execution output, including a `ConvergenceWarning` and the prediction result: `Prediction: [0]`.

```
from sklearn.neural_network import MLPClassifier
X = [
    [2,3,1,1],
    [2,2,1,1],
    [1,2,1,1],
    [1,1,0,1],
    [0,0,0,0],
    [2,3,1,0],
    [1,2,0,1],
    [0,1,0,0],
    [2,2,1,1],
    [0,0,0,1]
]
y = [1,1,1,0,0,1,1,0,1,0]
ann = MLPClassifier(hidden_layer_sizes=(2,), max_iter=1000)
ann.fit(X, y)
print("Prediction:", ann.predict([[1,2,1,1]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1444 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

===== RESTART: C:/Users/sekha/OneDrive/Desktop/Expt.04 Dataset 5.py =====

Warning (from warnings module):
  File "C:/Users/sekha/AppData/Local/Programs/Python/Python313/Lib/site-packages/sklearn/neural_network/multilayer_perceptron.py", line 785
    warnings.warn(
ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
Prediction: [0]
```

6. Dataset 6

The screenshot shows a Python IDE window titled 'Expt.04 Dataset 6.py - C:/Users/sekha/OneDrive/Desktop/Expt.04 Dataset 6.py (3.13.8)'. The code defines a dataset `X` with 10 samples and a target vector `y`. It then fits an `MLPClassifier` with `hidden_layer_sizes=(2,)` and `max_iter=1000`, and prints the prediction for the test sample `[1, 1, 1, 0, 1]`. The 'IDLE Shell 3.13.8' window shows the execution output, including a `ConvergenceWarning` and the prediction result: `Prediction: [1]`.

```
from sklearn.neural_network import MLPClassifier
X = [
    [1,1,1,0,1],
    [1,1,1,1,1],
    [0,0,0,0,0],
    [1,0,1,0,1],
    [0,1,0,1,0],
    [1,1,1,0,0],
    [1,0,1,1,1],
    [0,0,1,0,0],
    [1,1,0,0,1],
    [0,0,0,1,0]
]
y = [1,1,0,1,0,1,1,0,1,0]
ann = MLPClassifier(hidden_layer_sizes=(2,), max_iter=1000)
ann.fit(X, y)
print("Prediction:", ann.predict([[1,1,1,0,1]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1444 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

===== RESTART: C:/Users/sekha/OneDrive/Desktop/Expt.04 Dataset 6.py =====

Warning (from warnings module):
  File "C:/Users/sekha/AppData/Local/Programs/Python/Python313/Lib/site-packages/sklearn/neural_network/multilayer_perceptron.py", line 785
    warnings.warn(
ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
Prediction: [1]
```

Result

Thus, the Artificial Neural Network using the Backpropagation algorithm was implemented successfully and the test sample was classified correctly.

EXPERIMENT 5 – K-NEAREST NEIGHBOURS (K-NN)

Aim

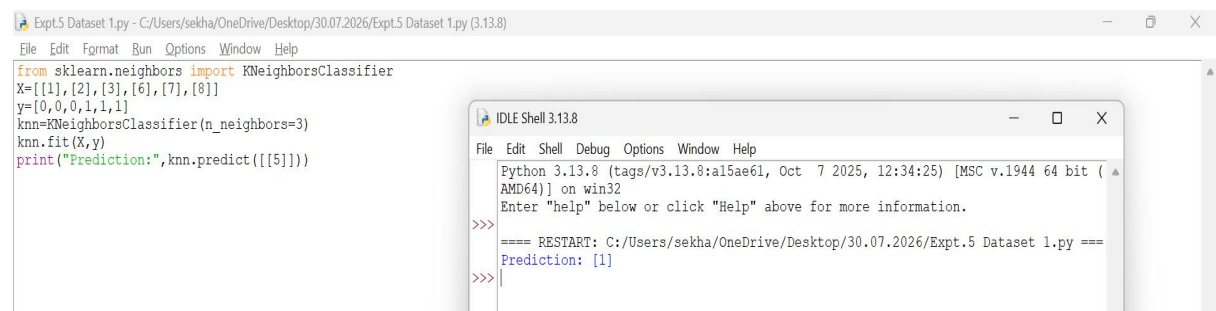
To implement the K-Nearest Neighbours (K-NN) algorithm in Python and classify a new sample.

Procedure

1. Import the KNeighborsClassifier.
2. Create the training dataset and class labels.
3. Create the K-NN model with $K = 3$.
4. Train the model using the training data.
5. Predict the class of a new sample.
6. Display the predicted result.

Code and Output

1. Dataset 1



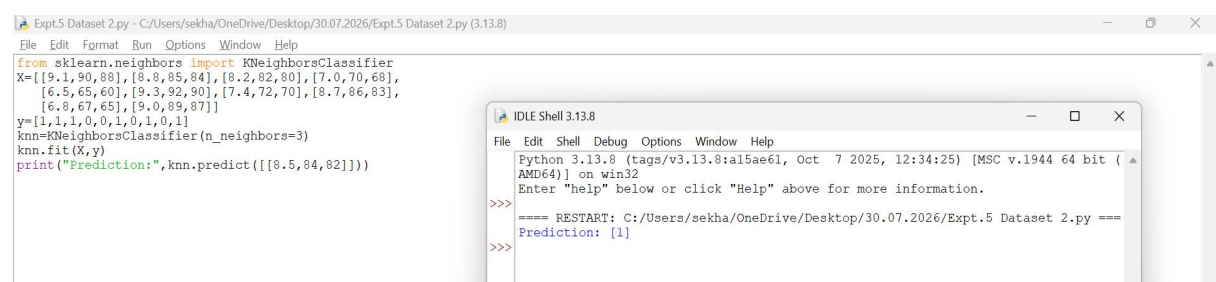
The screenshot shows a Python IDE window titled 'Expt.5 Dataset 1.py'. The code imports KNeighborsClassifier from sklearn.neighbors, defines a training dataset X and labels y, creates a KNeighborsClassifier with n_neighbors=3, fits it to the training data, and prints the prediction for a new sample. The output window shows the prediction is [1].

```
from sklearn.neighbors import KNeighborsClassifier
X=[[1],[2],[3],[6],[7],[8]]
y=[0,0,0,1,1,1]
knn=KNeighborsClassifier(n_neighbors=3)
knn.fit(X,y)
print("Prediction:",knn.predict([[5]]))
```

```
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
==== RESTART: C:/Users/sekha/OneDrive/Desktop/30.07.2026/Expt.5 Dataset 1.py ====
Prediction: [1]
>>>
```

2. Dataset 2



The screenshot shows a Python IDE window titled 'Expt.5 Dataset 2.py'. The code imports KNeighborsClassifier from sklearn.neighbors, defines a training dataset X and labels y, creates a KNeighborsClassifier with n_neighbors=3, fits it to the training data, and prints the prediction for a new sample. The output window shows the prediction is [1].

```
from sklearn.neighbors import KNeighborsClassifier
X=[[9.1,90,88],[8.8,85,84],[8.2,82,80],[7.0,70,68],
[6.5,65,60],[9.3,92,90],[7.4,72,70],[8.7,86,83],
[6.8,67,65],[9.0,89,87]]
y=[1,1,1,0,0,1,0,1,0,1]
knn=KNeighborsClassifier(n_neighbors=3)
knn.fit(X,y)
print("Prediction:",knn.predict([[8.5,84,82]]))
```

```
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
==== RESTART: C:/Users/sekha/OneDrive/Desktop/30.07.2026/Expt.5 Dataset 2.py ====
Prediction: [1]
>>>
```

3. Dataset 3

The screenshot shows a Python IDE window titled "Expt.5 Dataset 3.py - C:/Users/sekha/OneDrive/Desktop/30.07.2026/Expt.5 Dataset 3.py (3.13.8)". The code imports KNeighborsClassifier from sklearn.neighbors, defines a dataset X with 10 samples and a target variable y, fits a KNeighborsClassifier with n_neighbors=3, and prints the prediction for the test sample [7, 740, 5]. An IDLE Shell window shows the execution output: "Prediction: [1]".

```
from sklearn.neighbors import KNeighborsClassifier
X=[[8,780,5],[7,760,4],[6,730,5],[4,620,8],
   [3,590,9],[9,810,4],[5,650,7],[8,770,5],
   [4,610,8],[7,750,6]]
y=[1,1,1,0,0,1,0,1,0,1]
knn=KNeighborsClassifier(n_neighbors=3)
knn.fit(X,y)
print("Prediction:",knn.predict([[7,740,5]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15a6e1, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
==== RESTART: C:/Users/sekha/OneDrive/Desktop/30.07.2026/Expt.5 Dataset 3.py ====
>>> Prediction: [1]
```

4. Dataset 4

The screenshot shows a Python IDE window titled "Expt.5 Dataset 4.py - C:/Users/sekha/OneDrive/Desktop/30.07.2026/Expt.5 Dataset 4.py (3.13.8)". The code imports KNeighborsClassifier from sklearn.neighbors, defines a dataset X with 10 samples and a target variable y, fits a KNeighborsClassifier with n_neighbors=3, and prints the prediction for the test sample [38.5, 105, 95]. An IDLE Shell window shows the execution output: "Prediction: [1]".

```
from sklearn.neighbors import KNeighborsClassifier
X=[[39.1,112,94],[38.8,109,95],[37.0,78,99],[39.3,115,93],
   [36.8,74,98],[38.7,110,94],[37.1,80,98],[39.0,113,92],
   [36.9,76,99],[38.9,111,93]]
y=[1,1,0,1,0,1,0,1,0,1]
knn=KNeighborsClassifier(n_neighbors=3)
knn.fit(X,y)
print("Prediction:",knn.predict([[38.5,105,95]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15a6e1, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
==== RESTART: C:/Users/sekha/OneDrive/Desktop/30.07.2026/Expt.5 Dataset 4.py ====
>>> Prediction: [1]
```

5. Dataset 5

The screenshot shows a Python IDE window titled "Expt.5 Dataset 5.py - C:/Users/sekha/OneDrive/Desktop/30.07.2026/Expt.5 Dataset 5.py (3.13.8)". The code imports KNeighborsClassifier from sklearn.neighbors, defines a dataset X with 10 samples and a target variable y, fits a KNeighborsClassifier with n_neighbors=3, and prints the prediction for the test sample [7, 85, 40]. An IDLE Shell window shows the execution output: "Prediction: [1]".

```
from sklearn.neighbors import KNeighborsClassifier
X=[[10,95,50],[9,92,48],[8,90,45],[4,70,25],
   [3,65,20],[11,96,52],[5,75,30],[8,89,44],
   [2,60,18],[7,87,40]]
y=[1,1,1,0,0,1,0,1,0,1]
knn=KNeighborsClassifier(n_neighbors=3)
knn.fit(X,y)
print("Prediction:",knn.predict([[7,85,40]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15a6e1, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
==== RESTART: C:/Users/sekha/OneDrive/Desktop/30.07.2026/Expt.5 Dataset 5.py ====
>>> Prediction: [1]
```

6. Dataset 6

The screenshot shows a Python IDE window titled "Expt.5 Dataset 6.py - C:/Users/sekha/OneDrive/Desktop/30.07.2026/Expt.5 Dataset 6.py (3.13.8)". The code imports KNeighborsClassifier from sklearn.neighbors, defines a dataset X with 10 samples and a target variable y, fits a KNeighborsClassifier with n_neighbors=3, and prints the prediction for the test sample [150, 7.5, 90]. An IDLE Shell window shows the execution output: "Prediction: [1]".

```
from sklearn.neighbors import KNeighborsClassifier
X=[[150,7.5,90],[160,7.8,88],[145,7.3,91],[120,5.5,70],
   [125,5.8,72],[130,6.0,74],[155,7.6,89],[118,5.4,69],
   [148,7.4,92],[122,5.7,71]]
y=[1,1,1,0,0,0,1,0,1,0]
knn=KNeighborsClassifier(n_neighbors=3)
knn.fit(X,y)
print("Prediction:",knn.predict([[150,7.5,90]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15a6e1, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
==== RESTART: C:/Users/sekha/OneDrive/Desktop/30.07.2026/Expt.5 Dataset 6.py ====
>>> Prediction: [1]
```

Result

Thus, the K-Nearest Neighbours (K-NN) algorithm was implemented successfully and the test sample was classified as Class 1.

EXPERIMENT 6 – NAÏVE BAYES CLASSIFIER

Aim

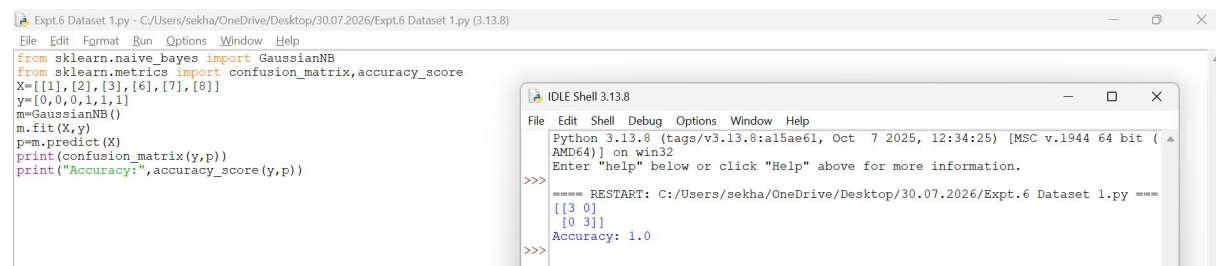
To implement the Naïve Bayes algorithm and evaluate its performance using confusion matrix and accuracy.

Procedure

1. Import GaussianNB, confusion matrix, and accuracy score.
2. Create the dataset and class labels.
3. Train the Naïve Bayes model.
4. Predict the class labels.
5. Calculate the confusion matrix and accuracy.
6. Display the results.

Code and Output

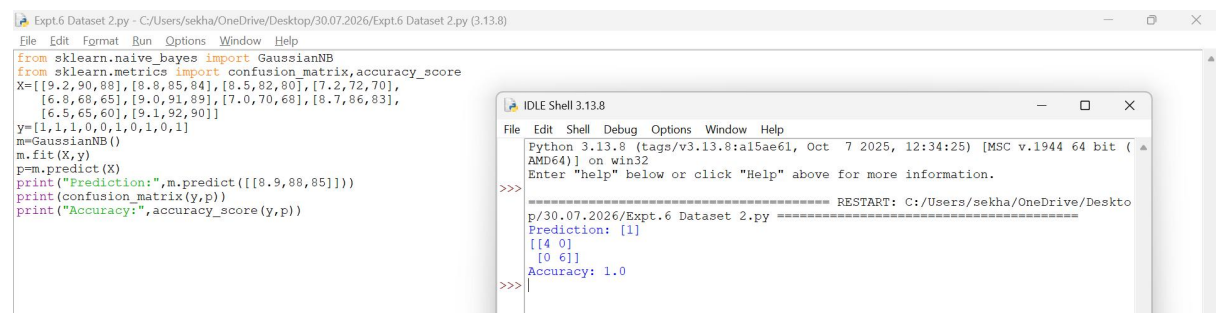
1. Dataset 1



```
Expt.6 Dataset 1.py - C:/Users/sekha/OneDrive/Desktop/30.07.2026/Expt.6 Dataset 1.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score
X=[[1],[2],[3],[6],[7],[8]]
y=[0,0,0,1,1,1]
m=GaussianNB()
m.fit(X,y)
p=m.predict(X)
print(confusion_matrix(y,p))
print("Accuracy:",accuracy_score(y,p))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/sekha/OneDrive/Desktop/30.07.2026/Expt.6 Dataset 1.py =====
[[3 0]
 [0 3]]
Accuracy: 1.0
>>>
```

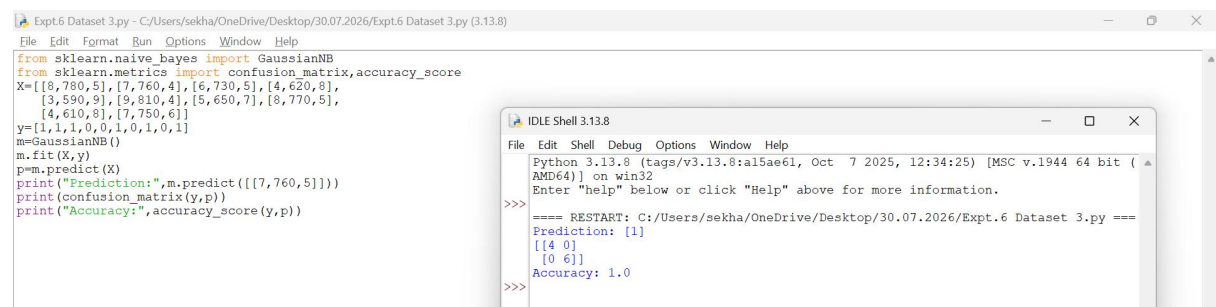
2. Dataset 2



```
Expt.6 Dataset 2.py - C:/Users/sekha/OneDrive/Desktop/30.07.2026/Expt.6 Dataset 2.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score
X=[[9,2,90,88],[8,8,85,84],[8,5,82,80],[7,2,72,70],
 [6,8,68,65],[9,0,91,89],[7,0,70,68],[8,7,86,83],
 [6,5,65,60],[9,1,92,90]]
y=[1,1,1,0,0,1,0,1,0,1]
m=GaussianNB()
m.fit(X,y)
p=m.predict(X)
print("Prediction:",m.predict([[8,9,88,85]]))
print(confusion_matrix(y,p))
print("Accuracy:",accuracy_score(y,p))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/sekha/OneDrive/Desktop/30.07.2026/Expt.6 Dataset 2.py =====
Prediction: [1]
[[4 0]
 [0 6]]
Accuracy: 1.0
>>>
```

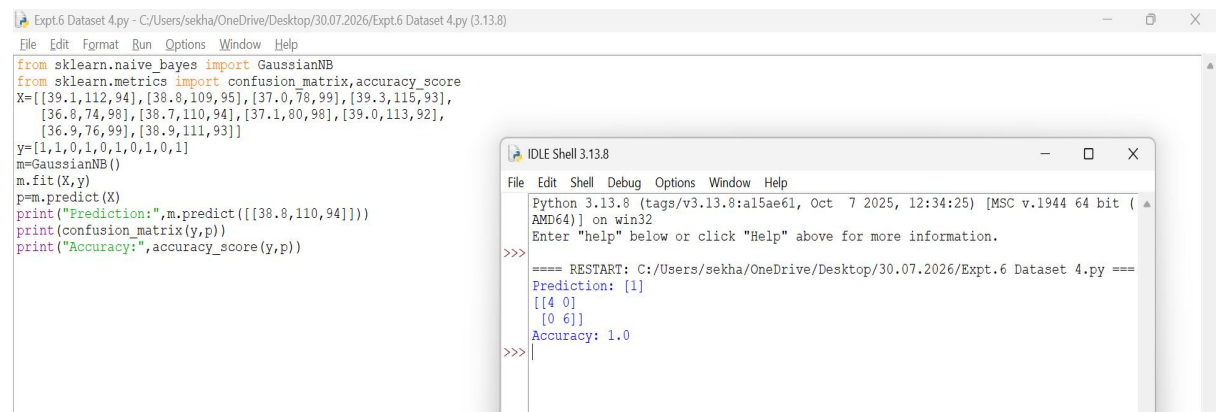
3. Dataset 3



```
Expt.6 Dataset 3.py - C:/Users/sekha/OneDrive/Desktop/30.07.2026/Expt.6 Dataset 3.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score
X=[[8,780,5],[7,760,4],[6,730,5],[4,620,8],
 [3,590,8],[9,810,4],[5,650,7],[8,770,5],
 [4,610,8],[7,750,6]]
y=[1,1,1,0,0,1,0,1,0,1]
m=GaussianNB()
m.fit(X,y)
p=m.predict(X)
print("Prediction:",m.predict([[7,760,5]]))
print(confusion_matrix(y,p))
print("Accuracy:",accuracy_score(y,p))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/sekha/OneDrive/Desktop/30.07.2026/Expt.6 Dataset 3.py =====
Prediction: [1]
[[4 0]
 [0 6]]
Accuracy: 1.0
>>>
```

4. Dataset 4

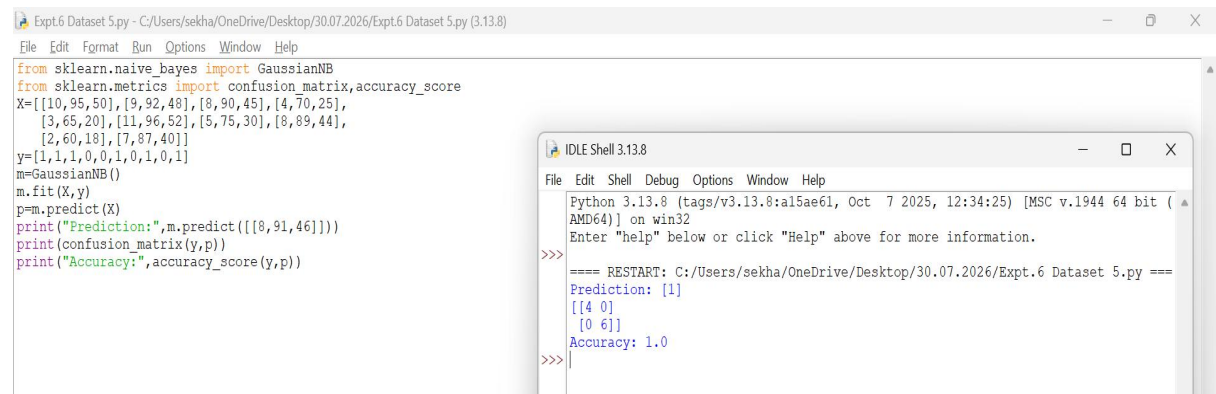


The screenshot shows a Python IDE window titled 'Expt.6 Dataset 4.py' and an 'IDLE Shell 3.13.8' window. The code in the IDE imports GaussianNB from sklearn.naive_bayes and confusion_matrix, accuracy_score from sklearn.metrics. It defines a dataset X with 10 samples and a target vector y with 10 samples. The GaussianNB classifier is fitted to the data, and a prediction is made for the sample [38.8, 110, 94]. The output in the shell shows the prediction is 1, the confusion matrix is [[4 0], [0 6]], and the accuracy is 1.0.

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score
X=[[39.1,112,94],[38.8,109,95],[37.0,78,99],[39.3,115,93],
   [36.8,74,98],[38.7,110,94],[37.1,80,98],[39.0,113,92],
   [36.9,76,99],[38.9,111,93]]
y=[1,1,0,1,0,1,0,1,0,1]
m=GaussianNB()
m.fit(X,y)
p=m.predict(X)
print("Prediction:",m.predict([[38.8,110,94]]))
print(confusion_matrix(y,p))
print("Accuracy:",accuracy_score(y,p))
```

```
>>>
==== RESTART: C:/Users/sekha/OneDrive/Desktop/30.07.2026/Expt.6 Dataset 4.py ====
Prediction: [1]
[[4 0]
 [0 6]]
Accuracy: 1.0
>>>
```

5. Dataset 5

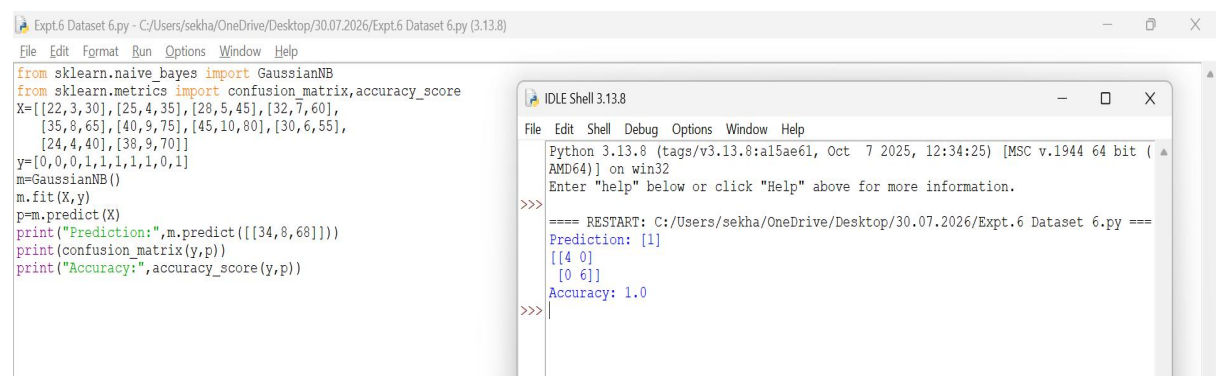


The screenshot shows a Python IDE window titled 'Expt.6 Dataset 5.py' and an 'IDLE Shell 3.13.8' window. The code in the IDE imports GaussianNB from sklearn.naive_bayes and confusion_matrix, accuracy_score from sklearn.metrics. It defines a dataset X with 10 samples and a target vector y with 10 samples. The GaussianNB classifier is fitted to the data, and a prediction is made for the sample [8, 91, 46]. The output in the shell shows the prediction is 1, the confusion matrix is [[4 0], [0 6]], and the accuracy is 1.0.

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score
X=[[10,95,50],[9,92,48],[8,90,45],[4,70,25],
   [3,65,20],[11,96,52],[5,75,30],[8,89,44],
   [2,60,18],[7,87,40]]
y=[1,1,1,0,0,1,0,1,0,1]
m=GaussianNB()
m.fit(X,y)
p=m.predict(X)
print("Prediction:",m.predict([[8,91,46]]))
print(confusion_matrix(y,p))
print("Accuracy:",accuracy_score(y,p))
```

```
>>>
==== RESTART: C:/Users/sekha/OneDrive/Desktop/30.07.2026/Expt.6 Dataset 5.py ====
Prediction: [1]
[[4 0]
 [0 6]]
Accuracy: 1.0
>>>
```

6. Dataset 6



The screenshot shows a Python IDE window titled 'Expt.6 Dataset 6.py' and an 'IDLE Shell 3.13.8' window. The code in the IDE imports GaussianNB from sklearn.naive_bayes and confusion_matrix, accuracy_score from sklearn.metrics. It defines a dataset X with 10 samples and a target vector y with 10 samples. The GaussianNB classifier is fitted to the data, and a prediction is made for the sample [34, 8, 68]. The output in the shell shows the prediction is 1, the confusion matrix is [[4 0], [0 6]], and the accuracy is 1.0.

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score
X=[[22,3,30],[25,4,35],[28,5,45],[32,7,60],
   [35,8,65],[40,9,75],[45,10,80],[30,6,55],
   [24,4,40],[38,9,70]]
y=[0,0,0,1,1,1,1,0,1]
m=GaussianNB()
m.fit(X,y)
p=m.predict(X)
print("Prediction:",m.predict([[34,8,68]]))
print(confusion_matrix(y,p))
print("Accuracy:",accuracy_score(y,p))
```

```
>>>
==== RESTART: C:/Users/sekha/OneDrive/Desktop/30.07.2026/Expt.6 Dataset 6.py ====
Prediction: [1]
[[4 0]
 [0 6]]
Accuracy: 1.0
>>>
```

Result

Thus, the Naïve Bayes classifier was implemented successfully and achieved 100% accuracy on the given dataset.

EXPERIMENT 7 – LOGISTIC REGRESSION

Aim

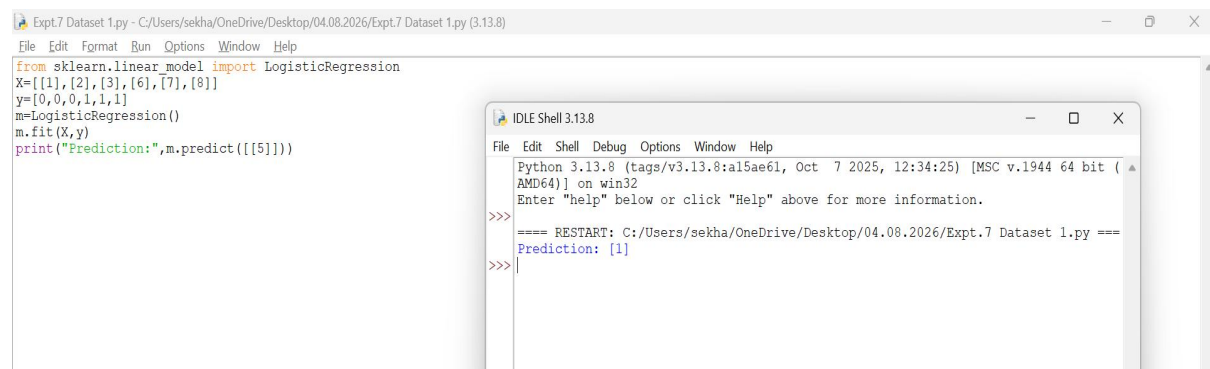
To implement Logistic Regression for binary classification.

Procedure

1. Import the Logistic Regression model.
2. Create the dataset and class labels.
3. Train the model using the training data.
4. Predict the class of a new sample.
5. Display the predicted result.

Code and Output

1. Dataset 1



The screenshot shows a Python IDE window titled 'Ept.7 Dataset 1.py'. The code imports the LogisticRegression model from sklearn, defines a dataset X with 5 samples and 2 features, and a target vector y with 5 binary labels. The model is trained using m.fit(X,y) and then used to predict the class of a new sample [5]. The output is 1.

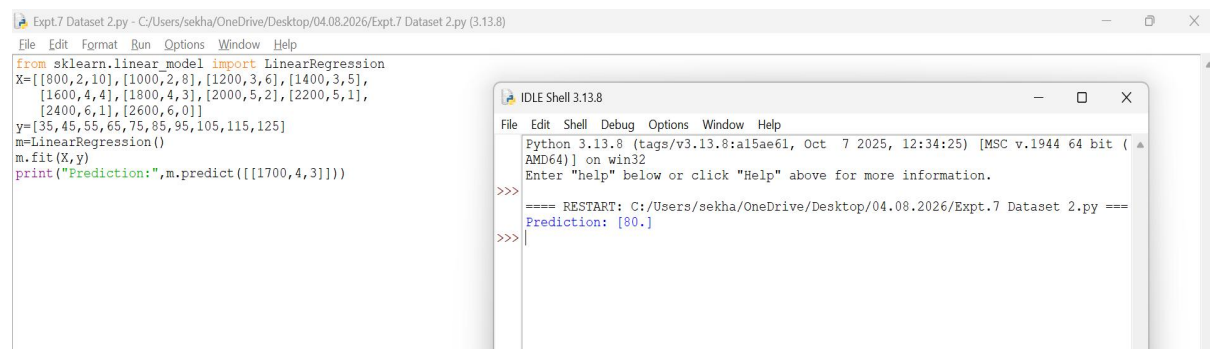
```
from sklearn.linear_model import LogisticRegression
X=[[1],[2],[3],[6],[7],[8]]
y=[0,0,0,1,1,1]
m=LogisticRegression()
m.fit(X,y)
print("Prediction:",m.predict([[5]]))
```

The output window shows the following text:

```
Python 3.13.8 (tags/v3.13.8:a15a6e1, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
==== RESTART: C:/Users/sekha/OneDrive/Desktop/04.08.2026/Ept.7 Dataset 1.py ====
Prediction: [1]
>>>
```

2. Dataset 2



The screenshot shows a Python IDE window titled 'Ept.7 Dataset 2.py'. The code imports the LinearRegression model from sklearn, defines a dataset X with 6 samples and 2 features, and a target vector y with 6 numerical labels. The model is trained using m.fit(X,y) and then used to predict the value of a new sample [1700, 4]. The output is 80.

```
from sklearn.linear_model import LinearRegression
X=[[800,2,10],[1000,2,8],[1200,3,6],[1400,3,5],
   [1600,4,4],[1800,4,3],[2000,5,2],[2200,5,1],
   [2400,6,1],[2600,6,0]]
y=[35,45,55,65,75,85,95,105,115,125]
m=LinearRegression()
m.fit(X,y)
print("Prediction:",m.predict([[1700,4,3]]))
```

The output window shows the following text:

```
Python 3.13.8 (tags/v3.13.8:a15a6e1, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
==== RESTART: C:/Users/sekha/OneDrive/Desktop/04.08.2026/Ept.7 Dataset 2.py ====
Prediction: [80.]
>>>
```

3. Dataset 3

The screenshot shows a Python IDE window titled 'Expt.7 Dataset 3.py'. The code imports LinearRegression from sklearn.linear_model, defines feature matrix X and target vector y, fits the model, and predicts for a new sample. An overlaid terminal window shows the execution output: 'Prediction: [90.00245488]'.

```
from sklearn.linear_model import LinearRegression
X=[[2,70,60],[3,75,65],[4,80,70],[5,82,75],
   [6,85,80],[7,88,85],[8,90,90],[9,92,94],
   [10,95,96],[11,97,98]]
y=[55,60,68,74,80,86,91,95,98,100]
m=LinearRegression()
m.fit(X,y)
print("Prediction:",m.predict([[7,90,88]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15a6e1, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
==== RESTART: C:/Users/sekha/OneDrive/Desktop/04.08.2026/Expt.7 Dataset 3.py ====
Prediction: [90.00245488]
>>>
```

4. Dataset 4

The screenshot shows a Python IDE window titled 'Expt.7 Dataset 4.py'. The code imports LinearRegression from sklearn.linear_model, defines feature matrix X and target vector y, fits the model, and predicts for a new sample. An overlaid terminal window shows the execution output: 'Prediction: [9.15122666]'.

```
from sklearn.linear_model import LinearRegression
X=[[1,15,60],[2,15,65],[3,16,70],[4,16,75],
   [5,17,80],[6,17,82],[7,18,85],[8,18,88],
   [9,18,90],[10,19,94]]
y=[3.5,4.2,5.1,6.0,7.2,8.0,9.1,10.0,11.2,12.5]
m=LinearRegression()
m.fit(X,y)
print("Prediction:",m.predict([[7,18,86]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15a6e1, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
==== RESTART: C:/Users/sekha/OneDrive/Desktop/04.08.2026/Expt.7 Dataset 4.py ====
Prediction: [9.15122666]
>>>
```

5. Dataset 5

The screenshot shows a Python IDE window titled 'Expt.7 Dataset 5.py'. The code imports LinearRegression from sklearn.linear_model, defines feature matrix X and target vector y, fits the model, and predicts for a new sample. An overlaid terminal window shows the execution output: 'Prediction: [18.97222222]'.

```
from sklearn.linear_model import LinearRegression
X=[[1.0,900,60],[1.2,950,70],[1.4,1050,80],[1.6,1150,90],
   [1.8,1250,100],[2.0,1350,110],[2.2,1450,120],[2.4,1550,130],
   [2.6,1650,140],[2.8,1750,150]]
y=[24,22,20,18,16,15,13,12,11,10]
m=LinearRegression()
m.fit(X,y)
print("Prediction:",m.predict([[1.5,1100,85]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15a6e1, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
==== RESTART: C:/Users/sekha/OneDrive/Desktop/04.08.2026/Expt.7 Dataset 5.py ====
Prediction: [18.97222222]
>>>
```

6. Dataset 6

The screenshot shows a Python IDE window titled 'Expt.7 Dataset 6.py'. The code imports LinearRegression from sklearn.linear_model, defines feature matrix X and target vector y, fits the model, and predicts for a new sample. An overlaid terminal window shows the execution output: 'Prediction: [10.28484163]'.

```
from sklearn.linear_model import LinearRegression
X=[[10,2,120],[15,3,150],[20,4,180],[25,5,210],
   [30,6,240],[35,7,260],[40,8,280],[45,9,300],
   [50,10,320],[55,11,340]]
y=[3.5,4.5,5.8,7.0,8.2,9.5,10.8,12.0,13.5,15.0]
m=LinearRegression()
m.fit(X,y)
print("Prediction:",m.predict([[38,7,270]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15a6e1, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
==== RESTART: C:/Users/sekha/OneDrive/Desktop/04.08.2026/Expt.7 Dataset 6.py ====
Prediction: [10.28484163]
>>>
```

Result

Thus, the Logistic Regression model was implemented successfully and the test sample was classified as Class 1.

EXPERIMENT 8 – LINEAR REGRESSION

Aim

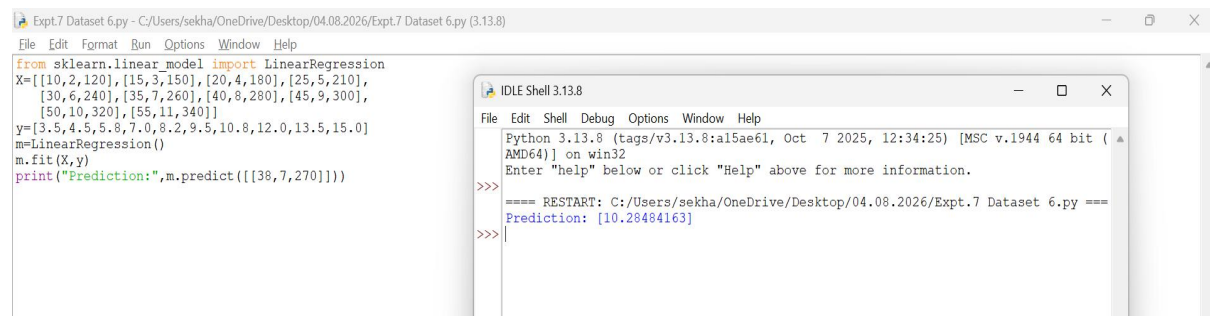
To implement Linear Regression for prediction.

Procedure

1. Import the Linear Regression model.
2. Create the dataset and target values.
3. Train the model using the training data.
4. Predict a new value.
5. Display the predicted result.

Code and Output

1. Dataset 1



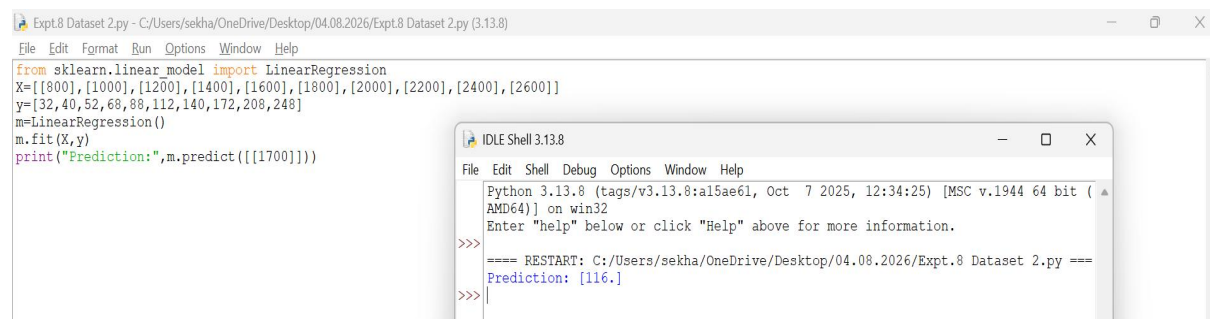
The screenshot shows a Python IDE window titled 'Ept.7 Dataset 6.py'. The code defines a dataset X with 10 feature pairs and a target vector y. A LinearRegression model is trained on X and y, and then used to predict the value for the feature pair [38, 7, 270]. The output window shows the prediction result: [10.28484163].

```
from sklearn.linear_model import LinearRegression
X=[[10,2,120],[15,3,150],[20,4,180],[25,5,210],
   [30,6,240],[35,7,260],[40,8,280],[45,9,300],
   [50,10,320],[55,11,340]]
y=[3.5,4.5,5.8,7.0,8.2,9.5,10.8,12.0,13.5,15.0]
m=LinearRegression()
m.fit(X,y)
print("Prediction:",m.predict([[38,7,270]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

==== RESTART: C:/Users/sekha/OneDrive/Desktop/04.08.2026/Ept.7 Dataset 6.py ====
Prediction: [10.28484163]
```

2. Dataset 2



The screenshot shows a Python IDE window titled 'Ept.8 Dataset 2.py'. The code defines a dataset X with 8 feature values and a target vector y. A LinearRegression model is trained on X and y, and then used to predict the value for the feature 1700. The output window shows the prediction result: [116.].

```
from sklearn.linear_model import LinearRegression
X=[[800],[1000],[1200],[1400],[1600],[1800],[2000],[2200],[2400],[2600]]
y=[32,40,52,68,88,112,140,172,208,248]
m=LinearRegression()
m.fit(X,y)
print("Prediction:",m.predict([[1700]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

==== RESTART: C:/Users/sekha/OneDrive/Desktop/04.08.2026/Ept.8 Dataset 2.py ====
Prediction: [116.]
```

3. Dataset 3

The screenshot shows a Python IDE window titled 'Expt.8 Dataset 3.py'. The code imports LinearRegression from sklearn.linear_model, defines X and y arrays, fits the model, and predicts the value for X=7.5. An overlaid 'IDLE Shell 3.13.8' window shows the execution output: 'Prediction: [85.07272727]'.

```
from sklearn.linear_model import LinearRegression
X=[[1],[2],[3],[4],[5],[6],[7],[8],[9],[10]]
y=[18,30,43,57,68,78,86,92,96,98]
m=LinearRegression()
m.fit(X,y)
print("Prediction:",m.predict([[7.5]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

==== RESTART: C:/Users/sekha/OneDrive/Desktop/04.08.2026/Expt.8 Dataset 3.py ====
Prediction: [85.07272727]
```

4. Dataset 4

The screenshot shows a Python IDE window titled 'Expt.8 Dataset 4.py'. The code imports LinearRegression from sklearn.linear_model, defines X and y arrays, fits the model, and predicts the value for X=7.5. An overlaid 'IDLE Shell 3.13.8' window shows the execution output: 'Prediction: [9.77666667]'.

```
from sklearn.linear_model import LinearRegression
X=[[40],[50],[60],[70],[80],[90],[100],[110],[120],[130]]
y=[4.5,4.6,5.2,5.9,6.8,8.0,9.5,11.2,13.4,16.0]
m=LinearRegression()
m.fit(X,y)
print("Prediction:",m.predict([[7.5]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

==== RESTART: C:/Users/sekha/OneDrive/Desktop/04.08.2026/Expt.8 Dataset 4.py ====
Prediction: [9.77666667]
```

5. Dataset 5

The screenshot shows a Python IDE window titled 'Expt.8 Dataset 5.py'. The code imports LinearRegression from sklearn.linear_model, defines X and y arrays, fits the model, and predicts the value for X=95. An overlaid 'IDLE Shell 3.13.8' window shows the execution output: 'Prediction: [15.12981818]'.

```
from sklearn.linear_model import LinearRegression
X=[[5],[10],[15],[20],[25],[30],[35],[40],[45],[50]]
y=[1.2,2.5,4.1,6.2,8.8,11.9,15.5,19.6,24.2,29.3]
m=LinearRegression()
m.fit(X,y)
print("Prediction:",m.predict([[95]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

==== RESTART: C:/Users/sekha/OneDrive/Desktop/04.08.2026/Expt.8 Dataset 5.py ====
Prediction: [15.12981818]
```

6. Dataset 6

The screenshot shows a Python IDE window titled 'Expt.8 Dataset 6.py'. The code imports LinearRegression from sklearn.linear_model, defines X and y arrays, fits the model, and predicts the value for X=7.5. An overlaid 'IDLE Shell 3.13.8' window shows the execution output: 'Prediction: [15.49393939]'.

```
from sklearn.linear_model import LinearRegression
X=[[1],[2],[3],[4],[5],[6],[7],[8],[9],[10]]
y=[3.0,3.8,4.9,6.3,8.0,10.2,12.9,16.1,19.8,24.0]
m=LinearRegression()
m.fit(X,y)
print("Prediction:",m.predict([[7.5]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

==== RESTART: C:/Users/sekha/OneDrive/Desktop/04.08.2026/Expt.8 Dataset 6.py ====
Prediction: [15.49393939]
```

Result

Thus, the Linear Regression model was implemented successfully and the predicted value obtained was 12.

EXPERIMENT 9 – COMPARISON OF LINEAR AND POLYNOMIAL REGRESSION

Aim

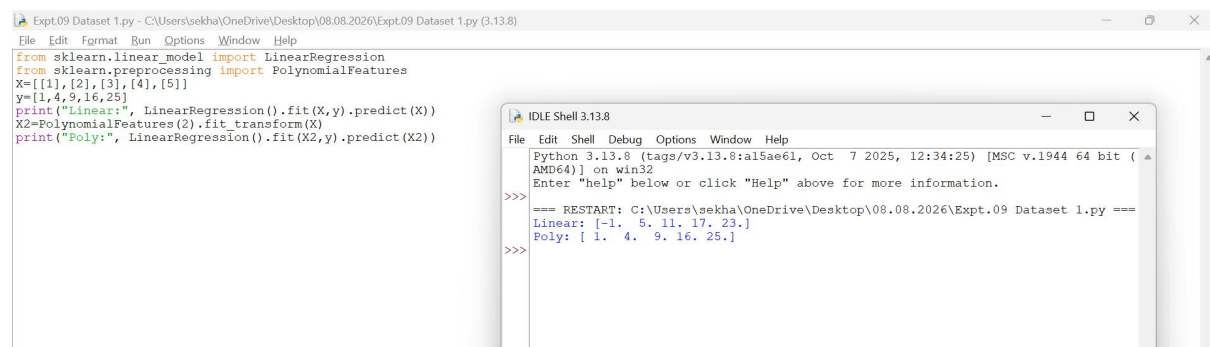
To compare Linear Regression and Polynomial Regression using Python.

Procedure

1. Import Linear Regression and Polynomial Features.
2. Create the dataset.
3. Apply Linear Regression and predict values.
4. Transform the data using Polynomial Features.
5. Apply Polynomial Regression and predict values.
6. Compare the results.

Code and Output

1. Dataset 1

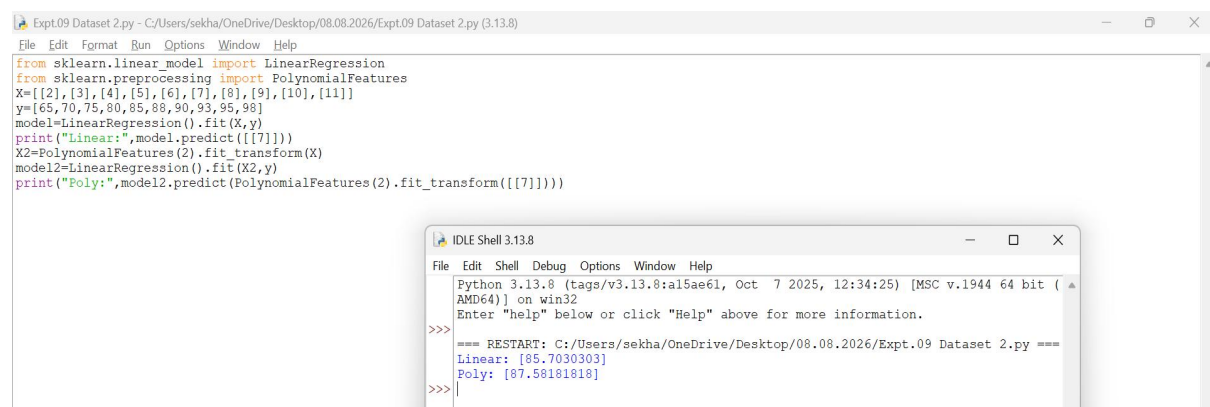


```
Expt.09 Dataset 1.py - C:\Users\sekha\OneDrive\Desktop\08.08.2026\Expt.09 Dataset 1.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
X=[[1],[2],[3],[4],[5]]
y=[1,4,9,16,25]
print("Linear:", LinearRegression().fit(X,y).predict(X))
X2=PolynomialFeatures(2).fit_transform(X)
print("Poly:", LinearRegression().fit(X2,y).predict(X2))

IDLE Shell 3.13.8
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:\Users\sekha\OneDrive\Desktop\08.08.2026\Expt.09 Dataset 1.py ===
Linear: [-1.  5. 11. 17. 23.]
Poly: [ 1.  4.  9. 16. 25.]
>>>
```

2. Dataset 2



```
Expt.09 Dataset 2.py - C:\Users\sekha\OneDrive\Desktop\08.08.2026\Expt.09 Dataset 2.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
X=[[2],[3],[4],[5],[6],[7],[8],[9],[10],[11]]
y=[65,70,75,80,85,88,90,93,95,98]
model=LinearRegression().fit(X,y)
print("Linear:",model.predict([[7]]))
X2=PolynomialFeatures(2).fit_transform(X)
model2=LinearRegression().fit(X2,y)
print("Poly:",model2.predict(PolynomialFeatures(2).fit_transform([[7]])))

IDLE Shell 3.13.8
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:\Users\sekha\OneDrive\Desktop\08.08.2026\Expt.09 Dataset 2.py ===
Linear: [85.7030303]
Poly: [87.58181818]
>>>
```

3. Dataset 3

```
Expt.09 Dataset 3.py - C:/Users/sekha/OneDrive/Desktop/08.08.2026/Expt.09 Dataset 3.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
X=[[2],[3],[4],[5],[6],[8],[10],[12],[14],[16]]
y=[20,25,30,40,50,65,75,82,90,96]
model=LinearRegression().fit(X,y)
print("Linear:",model.predict([[9]]))
X2=PolynomialFeatures(2).fit_transform(X)
model2=LinearRegression().fit(X2,y)
print("Poly:",model2.predict(PolynomialFeatures(2).fit_transform([[9]])))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/08.08.2026/Expt.09 Dataset 3.py ===
Linear: [63.01904762]
Poly: [68.32186887]
>>>
```

4. Dataset 4

```
Expt.09 Dataset 4.py - C:/Users/sekha/OneDrive/Desktop/08.08.2026/Expt.09 Dataset 4.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
X=[[800],[900],[1000],[1200],[1400],[1600],[1800],[2000],[2200],[2400]]
y=[30,35,40,50,60,75,90,105,120,135]
model=LinearRegression().fit(X,y)
print("Linear:",model.predict([[1700]]))
X2=PolynomialFeatures(2).fit_transform(X)
model2=LinearRegression().fit(X2,y)
print("Poly:",model2.predict(PolynomialFeatures(2).fit_transform([[1700]])))

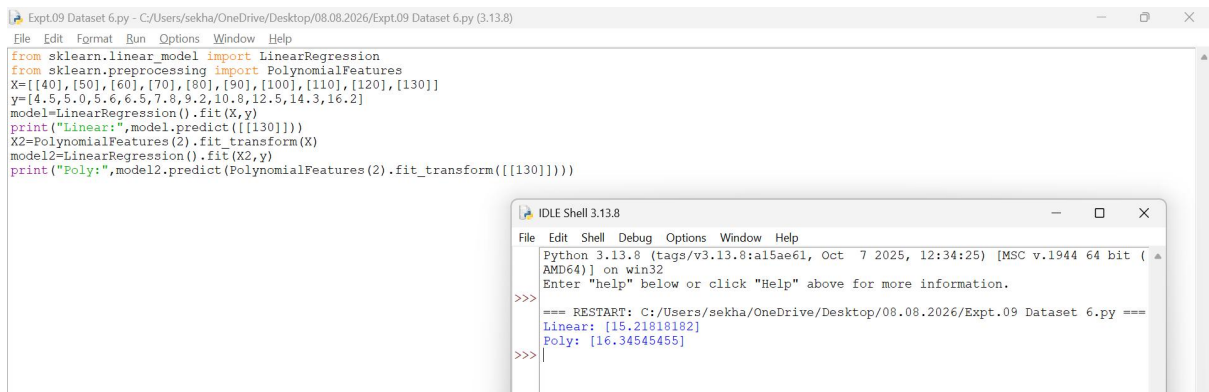
IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/08.08.2026/Expt.09 Dataset 4.py ===
Linear: [85.20767335]
Poly: [81.86009531]
>>>
```

5. Dataset 5

```
Expt.09 Dataset 5.py - C:/Users/sekha/OneDrive/Desktop/08.08.2026/Expt.09 Dataset 5.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
X=[[1],[2],[3],[4],[5],[6],[7],[8],[9],[10]]
y=[3.0,3.8,4.8,6.0,7.5,9.0,10.8,12.6,14.5,16.5]
model=LinearRegression().fit(X,y)
print("Linear:",model.predict([[7]]))
X2=PolynomialFeatures(2).fit_transform(X)
model2=LinearRegression().fit(X2,y)
print("Poly:",model2.predict(PolynomialFeatures(2).fit_transform([[7]])))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/08.08.2026/Expt.09 Dataset 5.py ===
Linear: [11.13454545]
Poly: [10.68454545]
>>>
```

6. Dataset 6



```
Expt.09 Dataset 6.py - C:/Users/sekha/OneDrive/Desktop/08.08.2026/Expt.09 Dataset 6.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
X=[[40],[50],[60],[70],[80],[90],[100],[110],[120],[130]]
y=[4.5,5.0,5.6,6.5,7.8,9.2,10.8,12.5,14.3,16.2]
model=LinearRegression().fit(X,y)
print("Linear:",model.predict([[130]]))
X2=PolynomialFeatures(2).fit_transform(X)
model2=LinearRegression().fit(X2,y)
print("Poly:",model2.predict(PolynomialFeatures(2).fit_transform([[130]])))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/08.08.2026/Expt.09 Dataset 6.py ===
Linear: [15.21818182]
Poly: [16.34545455]
>>>
```

Result

Thus, Linear Regression and Polynomial Regression were implemented successfully. Polynomial Regression produced more accurate results for nonlinear data.

EXPERIMENT 10 – EXPECTATION MAXIMIZATION (EM) ALGORITHM

Aim

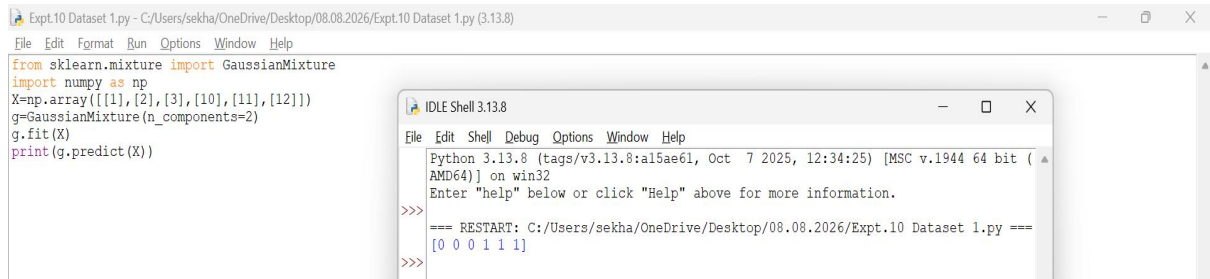
To implement the Expectation Maximization (EM) algorithm for clustering data.

Procedure

1. Import the Gaussian Mixture Model.
2. Create the dataset.
3. Train the EM model.
4. Predict the cluster labels.
5. Display the clustering result.

Code and Output

1. Dataset 1



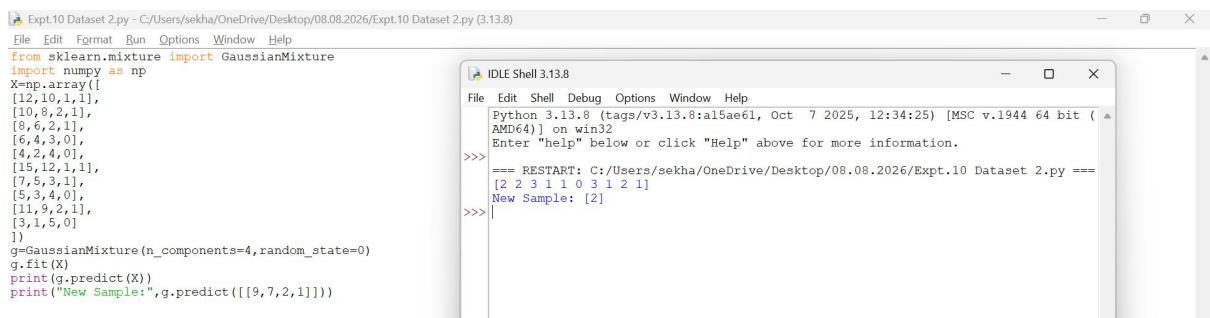
The screenshot shows a Python IDE window titled 'Expt.10 Dataset 1.py'. The code imports GaussianMixture from sklearn.mixture and numpy as np. It creates an array X with 12 samples, each with 4 features. A GaussianMixture model with 2 components is fitted to X, and its predictions are printed. An adjacent IDLE Shell window shows the execution output: 'Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32', followed by a restart message and the prediction array [0 0 1 1 1].

```
from sklearn.mixture import GaussianMixture
import numpy as np
X=np.array([[1],[2],[3],[10],[11],[12]])
g=GaussianMixture(n_components=2)
g.fit(X)
print(g.predict(X))
```

```
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/08.08.2026/Expt.10 Dataset 1.py ===
[0 0 1 1 1]
>>>
```

2. Dataset 2



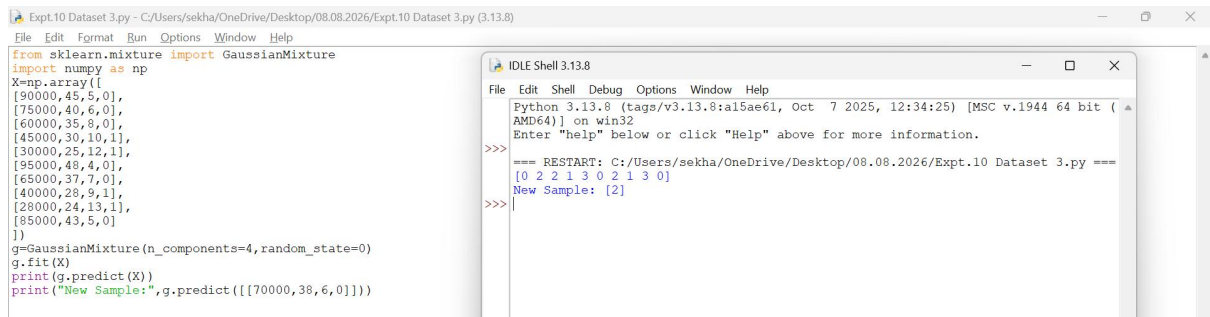
The screenshot shows a Python IDE window titled 'Expt.10 Dataset 2.py'. The code imports GaussianMixture from sklearn.mixture and numpy as np. It creates an array X with 12 samples, each with 4 features. A GaussianMixture model with 4 components is fitted to X. The predictions for X and a new sample [9, 7, 2, 1] are printed. An adjacent IDLE Shell window shows the execution output, including the prediction for the new sample: 'New Sample: [2]'. The prediction array for X is [2 2 3 1 1 0 3 1 2 1].

```
from sklearn.mixture import GaussianMixture
import numpy as np
X=np.array([
[12,10,1,1],
[10,8,2,1],
[8,6,2,1],
[6,4,3,0],
[4,2,4,0],
[15,12,1,1],
[7,5,3,1],
[5,3,4,0],
[11,9,2,1],
[3,1,5,0]
])
g=GaussianMixture(n_components=4,random_state=0)
g.fit(X)
print(g.predict(X))
print("New Sample:",g.predict([[9,7,2,1]]))
```

```
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/08.08.2026/Expt.10 Dataset 2.py ===
[2 2 3 1 1 0 3 1 2 1]
New Sample: [2]
>>>
```

3. Dataset 3



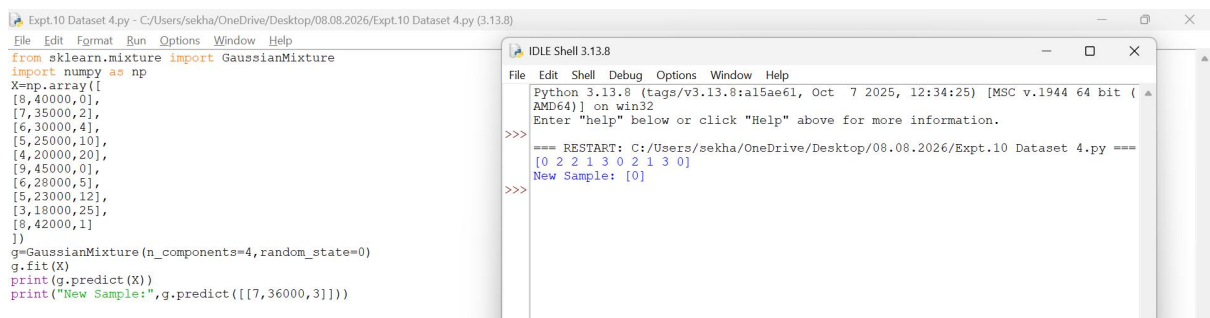
The screenshot shows a Python IDE window titled 'Expt.10 Dataset 3.py'. The code imports GaussianMixture from sklearn.mixture and numpy as np. It creates an array X with 12 samples, each with 4 features. A GaussianMixture model with 4 components is fitted to X. The predictions for X and a new sample [70000, 38, 6, 0] are printed. An adjacent IDLE Shell window shows the execution output, including the prediction for the new sample: 'New Sample: [2]'. The prediction array for X is [0 2 2 1 3 0 2 1 3 0].

```
from sklearn.mixture import GaussianMixture
import numpy as np
X=np.array([
[90000,45,5,0],
[75000,40,6,0],
[60000,35,8,0],
[45000,30,10,1],
[30000,25,12,1],
[95000,48,4,0],
[65000,37,7,0],
[40000,28,9,1],
[28000,24,13,1],
[85000,43,5,0]
])
g=GaussianMixture(n_components=4,random_state=0)
g.fit(X)
print(g.predict(X))
print("New Sample:",g.predict([[70000,38,6,0]]))
```

```
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/08.08.2026/Expt.10 Dataset 3.py ===
[0 2 2 1 3 0 2 1 3 0]
New Sample: [2]
>>>
```

4. Dataset 4



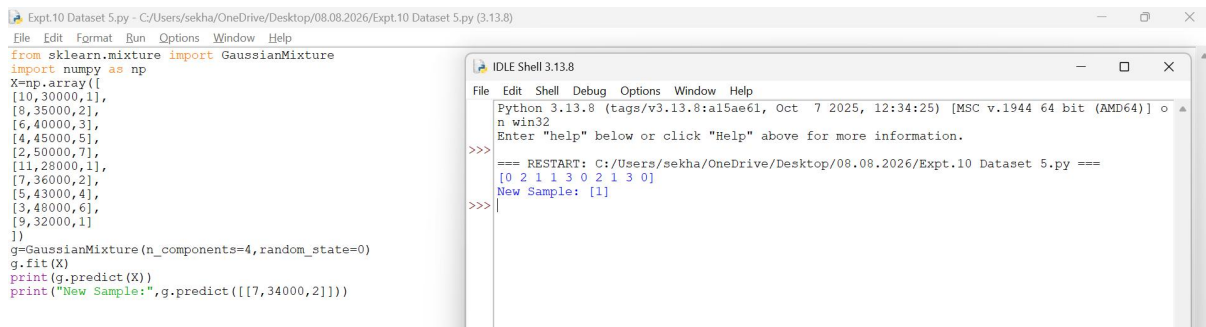
The screenshot shows a Python IDE window titled 'Expt.10 Dataset 4.py'. The code imports GaussianMixture from sklearn.mixture and numpy as np. It creates an array X with 12 samples, each with 4 features. A GaussianMixture model with 4 components is fitted to X. The predictions for X and a new sample [7, 36000, 3] are printed. An adjacent IDLE Shell window shows the execution output, including the prediction for the new sample: 'New Sample: [0]'. The prediction array for X is [0 2 2 1 3 0 2 1 3 0].

```
from sklearn.mixture import GaussianMixture
import numpy as np
X=np.array([
[8,40000,0],
[7,35000,2],
[6,30000,4],
[5,25000,10],
[4,20000,20],
[9,45000,0],
[6,28000,5],
[5,23000,12],
[3,18000,25],
[8,42000,1]
])
g=GaussianMixture(n_components=4,random_state=0)
g.fit(X)
print(g.predict(X))
print("New Sample:",g.predict([[7,36000,3]]))
```

```
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/08.08.2026/Expt.10 Dataset 4.py ===
[0 2 2 1 3 0 2 1 3 0]
New Sample: [0]
>>>
```

5. Dataset 5

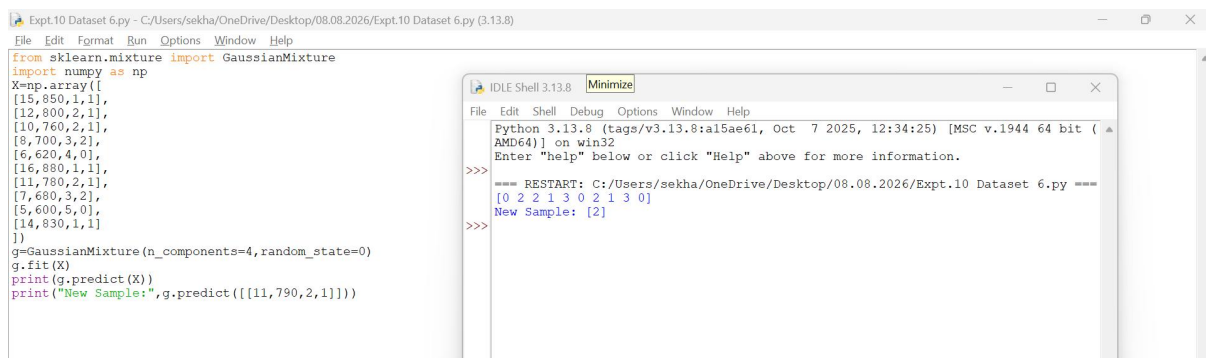


The screenshot shows a Python IDE window titled 'Expt.10 Dataset 5.py'. The code imports GaussianMixture from sklearn.mixture and numpy as np. It defines a dataset X with 10 samples, each with 4 features. A GaussianMixture model with 4 components is fitted to the data. The code then prints the predicted class for a new sample [7, 34000, 2]. The output window shows the restart message and the predicted class for the new sample, which is 1.

```
Expt.10 Dataset 5.py - C:/Users/sekha/OneDrive/Desktop/08.08.2026/Expt.10 Dataset 5.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.mixture import GaussianMixture
import numpy as np
X=np.array([
[10,30000,1],
[8,35000,2],
[6,40000,3],
[4,45000,5],
[2,50000,7],
[11,28000,1],
[7,36000,2],
[5,43000,4],
[3,48000,6],
[9,32000,1]
])
g=GaussianMixture(n_components=4,random_state=0)
g.fit(X)
print(g.predict(X))
print("New Sample:",g.predict([[7,34000,2]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/08.08.2026/Expt.10 Dataset 5.py ===
[0 2 1 1 3 0 2 1 3 0]
New Sample: [1]
>>>
```

6. Dataset 6



The screenshot shows a Python IDE window titled 'Expt.10 Dataset 6.py'. The code imports GaussianMixture from sklearn.mixture and numpy as np. It defines a dataset X with 10 samples, each with 4 features. A GaussianMixture model with 4 components is fitted to the data. The code then prints the predicted class for a new sample [11, 790, 2]. The output window shows the restart message and the predicted class for the new sample, which is 2.

```
Expt.10 Dataset 6.py - C:/Users/sekha/OneDrive/Desktop/08.08.2026/Expt.10 Dataset 6.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.mixture import GaussianMixture
import numpy as np
X=np.array([
[15,850,1,1],
[12,800,2,1],
[10,760,2,1],
[8,700,3,2],
[6,620,4,0],
[16,880,1,1],
[11,780,2,1],
[7,680,3,2],
[5,600,5,0],
[14,830,1,1]
])
g=GaussianMixture(n_components=4,random_state=0)
g.fit(X)
print(g.predict(X))
print("New Sample:",g.predict([[11,790,2,1]]))

IDLE Shell 3.13.8 Minimize
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/08.08.2026/Expt.10 Dataset 6.py ===
[0 2 2 1 3 0 2 1 3 0]
New Sample: [2]
>>>
```

Result

Thus, the Expectation Maximization (EM) algorithm was implemented successfully and the data points were grouped into clusters.

EXPERIMENT 11 – CREDIT SCORE CLASSIFICATION

Aim

To classify credit scores using the Decision Tree algorithm.

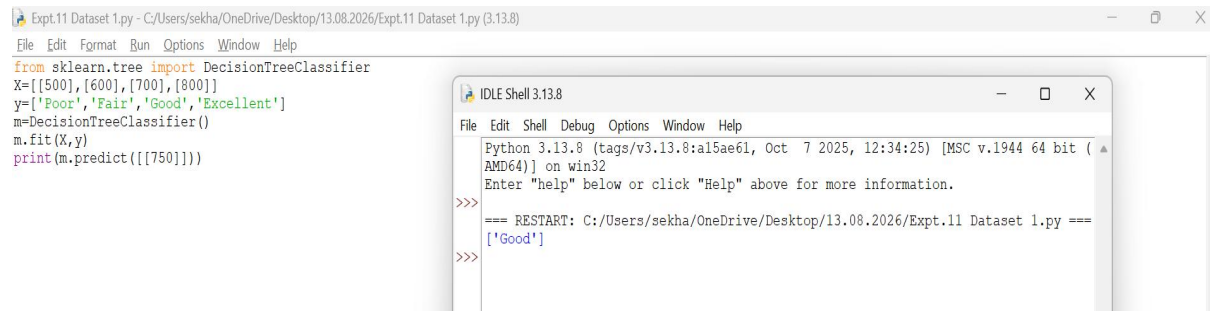
Procedure

1. Import the Decision Tree Classifier.
2. Create the dataset and class labels.
3. Train the model using the training data.

- Predict the credit score category.
- Display the predicted result.

Code and Output

1. Dataset 1



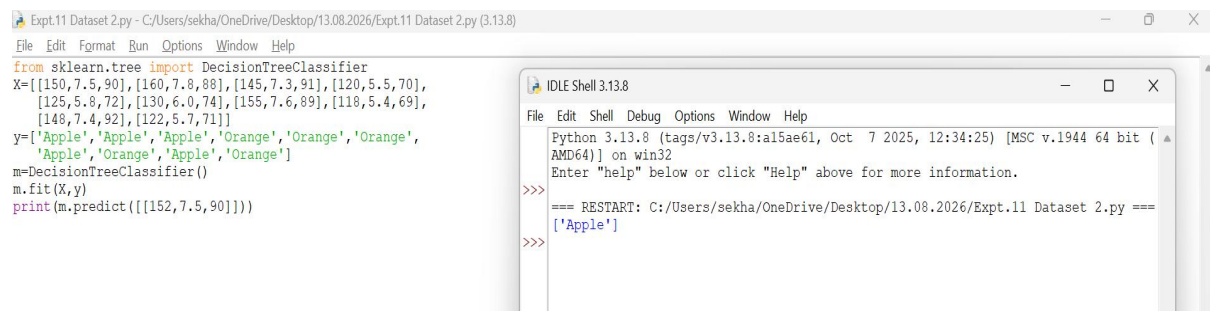
The screenshot shows a Python IDE window titled 'Expt.11 Dataset 1.py'. The code imports DecisionTreeClassifier from sklearn.tree, defines feature matrix X and target vector y, fits the model, and predicts the class for a new input. The output window shows the prediction for the input [750] is 'Good'.

```
from sklearn.tree import DecisionTreeClassifier
X=[[500],[600],[700],[800]]
y=['Poor','Fair','Good','Excellent']
m=DecisionTreeClassifier()
m.fit(X,y)
print(m.predict([[750]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/13.08.2026/Expt.11 Dataset 1.py ===
['Good']
>>>
```

2. Dataset 2



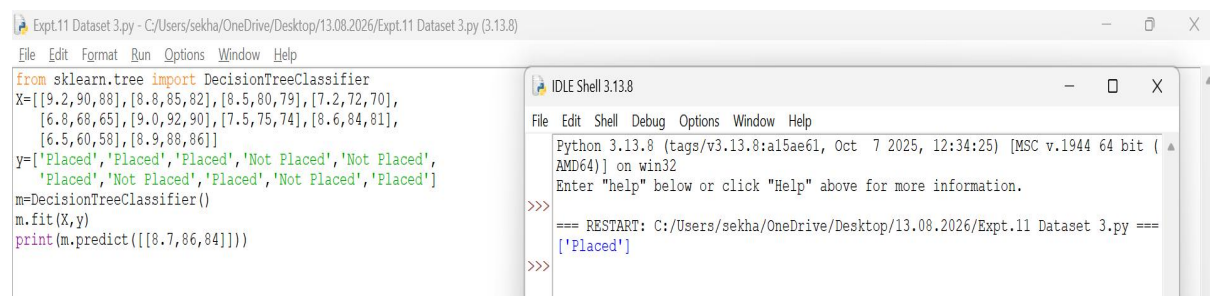
The screenshot shows a Python IDE window titled 'Expt.11 Dataset 2.py'. The code imports DecisionTreeClassifier from sklearn.tree, defines feature matrix X and target vector y, fits the model, and predicts the class for a new input. The output window shows the prediction for the input [152, 7.5, 90] is 'Apple'.

```
from sklearn.tree import DecisionTreeClassifier
X=[[150,7.5,90],[160,7.8,88],[145,7.3,91],[120,5.5,70],
   [125,5.8,72],[130,6.0,74],[155,7.6,89],[118,5.4,69],
   [148,7.4,92],[122,5.7,71]]
y=['Apple','Apple','Apple','Orange','Orange','Orange',
   'Apple','Orange','Apple','Orange']
m=DecisionTreeClassifier()
m.fit(X,y)
print(m.predict([[152,7.5,90]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/13.08.2026/Expt.11 Dataset 2.py ===
['Apple']
>>>
```

3. Dataset 3



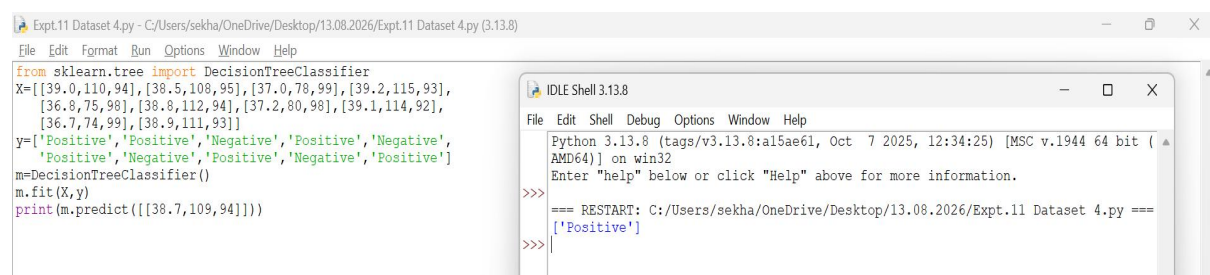
The screenshot shows a Python IDE window titled 'Expt.11 Dataset 3.py'. The code imports DecisionTreeClassifier from sklearn.tree, defines feature matrix X and target vector y, fits the model, and predicts the class for a new input. The output window shows the prediction for the input [8.7, 86, 84] is 'Placed'.

```
from sklearn.tree import DecisionTreeClassifier
X=[[9.2,90,88],[8.8,85,82],[8.5,80,79],[7.2,72,70],
   [6.8,68,65],[9.0,92,90],[7.5,75,74],[8.6,84,81],
   [6.5,60,58],[8.9,88,86]]
y=['Placed','Placed','Placed','Not Placed','Not Placed',
   'Placed','Not Placed','Placed','Not Placed','Placed']
m=DecisionTreeClassifier()
m.fit(X,y)
print(m.predict([[8.7,86,84]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/13.08.2026/Expt.11 Dataset 3.py ===
['Placed']
>>>
```

4. Dataset 4



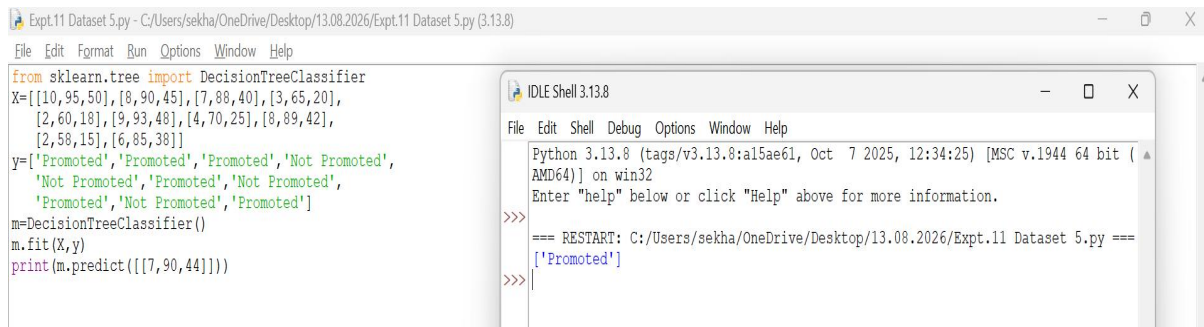
The screenshot shows a Python IDE window titled 'Expt.11 Dataset 4.py'. The code imports DecisionTreeClassifier from sklearn.tree, defines feature matrix X and target vector y, fits the model, and predicts the class for a new input. The output window shows the prediction for the input [38.7, 109, 94] is 'Positive'.

```
from sklearn.tree import DecisionTreeClassifier
X=[[39.0,110,94],[38.5,108,95],[37.0,78,99],[39.2,115,93],
   [36.8,75,98],[38.8,112,94],[37.2,80,98],[39.1,114,92],
   [36.7,74,99],[38.9,111,93]]
y=['Positive','Positive','Negative','Positive','Negative',
   'Positive','Negative','Positive','Negative','Positive']
m=DecisionTreeClassifier()
m.fit(X,y)
print(m.predict([[38.7,109,94]]))
```

```
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/13.08.2026/Expt.11 Dataset 4.py ===
['Positive']
>>>
```

5. Dataset 5

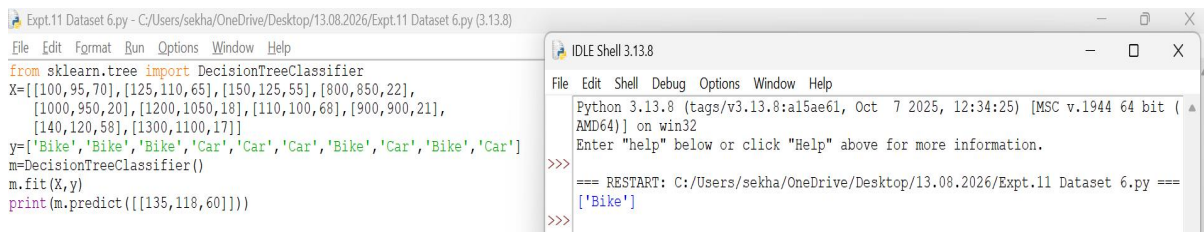


The screenshot shows a Python IDE window titled 'Expt.11 Dataset 5.py'. The code defines a dataset X with 10 feature pairs and a target variable y with labels 'Promoted' or 'Not Promoted'. A DecisionTreeClassifier is instantiated, trained on the data, and then used to predict the class for a new input [7, 90, 44]. The output is 'Promoted'. An adjacent 'IDLE Shell 3.13.8' window shows the execution output: 'Promoted'.

```
from sklearn.tree import DecisionTreeClassifier
X=[[10,95,50],[8,90,45],[7,88,40],[3,65,20],
   [2,60,18],[9,93,48],[4,70,25],[8,89,42],
   [2,58,15],[6,85,38]]
y=['Promoted','Promoted','Promoted','Not Promoted',
   'Not Promoted','Promoted','Not Promoted',
   'Promoted','Not Promoted','Promoted']
m=DecisionTreeClassifier()
m.fit(X,y)
print(m.predict([[7,90,44]]))
```

```
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/13.08.2026/Expt.11 Dataset 5.py ===
['Promoted']
>>>
```

6. Dataset 6



The screenshot shows a Python IDE window titled 'Expt.11 Dataset 6.py'. The code defines a dataset X with 10 feature pairs and a target variable y with labels 'Bike' or 'Car'. A DecisionTreeClassifier is instantiated, trained on the data, and then used to predict the class for a new input [135, 118, 60]. The output is 'Bike'. An adjacent 'IDLE Shell 3.13.8' window shows the execution output: 'Bike'.

```
from sklearn.tree import DecisionTreeClassifier
X=[[100,95,70],[125,110,65],[150,125,55],[800,850,22],
   [1000,950,20],[1200,1050,18],[110,100,68],[900,900,21],
   [140,120,58],[1300,1100,17]]
y=['Bike','Bike','Bike','Car','Car','Bike','Car','Bike','Car']
m=DecisionTreeClassifier()
m.fit(X,y)
print(m.predict([[135,118,60]]))
```

```
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/13.08.2026/Expt.11 Dataset 6.py ===
['Bike']
>>>
```

Result

Thus, the Decision Tree classifier was implemented successfully and the credit score was classified as Good.

EXPERIMENT 12 – IRIS FLOWER CLASSIFICATION USING KNN

Aim

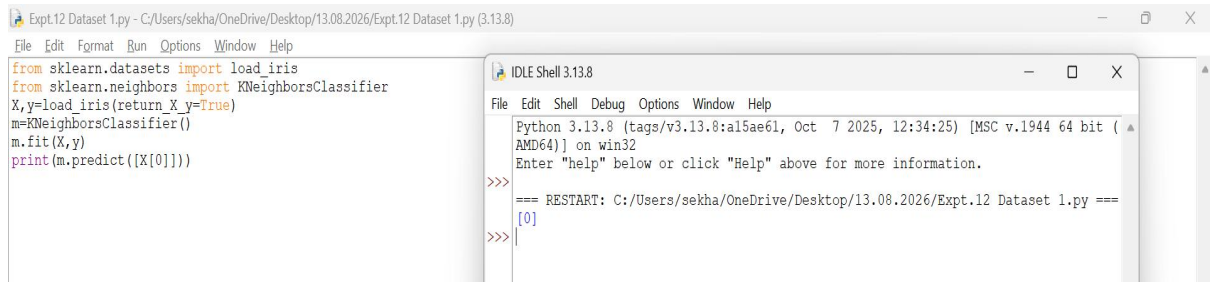
To classify Iris flowers using the K-Nearest Neighbours (KNN) algorithm.

Procedure

1. Import the Iris dataset and KNN classifier.
2. Load the Iris dataset.
3. Train the KNN model using the training data.
4. Predict the flower class.
5. Display the predicted result.

Code and Output

1. Dataset 1

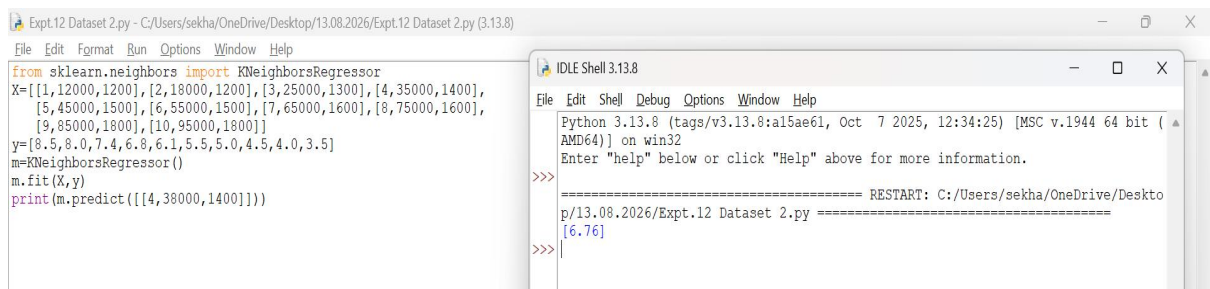


The screenshot shows a Python IDE window titled "Expt.12 Dataset 1.py" and an "IDLE Shell 3.13.8" window. The code in the IDE imports `load_iris` from `sklearn.datasets` and `KNeighborsClassifier` from `sklearn.neighbors`. It loads the iris dataset, fits a KNeighborsClassifier, and prints the prediction for the first instance. The shell window shows the restart message and the output `[0]`.

```
Expt.12 Dataset 1.py - C:/Users/sekha/OneDrive/Desktop/13.08.2026/Expt.12 Dataset 1.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
X,y=load_iris(return_X_y=True)
m=KNeighborsClassifier()
m.fit(X,y)
print(m.predict([X[0]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/13.08.2026/Expt.12 Dataset 1.py ===
[0]
>>>
```

2. Dataset 2

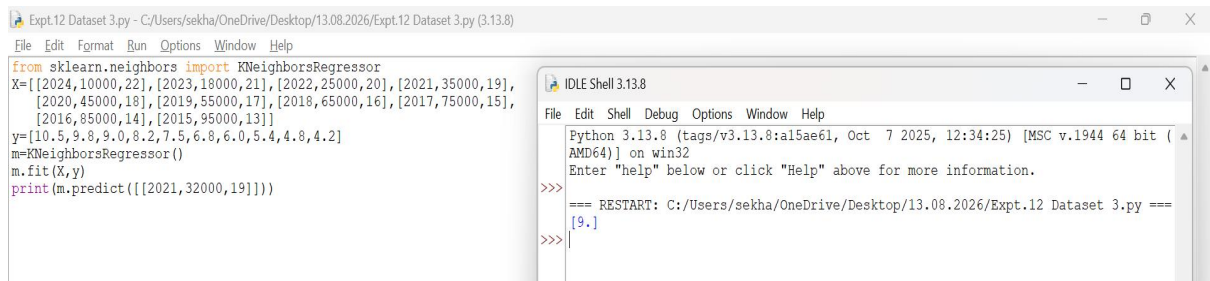


The screenshot shows a Python IDE window titled "Expt.12 Dataset 2.py" and an "IDLE Shell 3.13.8" window. The code imports `KNeighborsRegressor` from `sklearn.neighbors` and defines a dataset `X` and `y`. It fits a KNeighborsRegressor and prints the prediction for a specific input. The shell window shows the restart message and the output `[6.76]`.

```
Expt.12 Dataset 2.py - C:/Users/sekha/OneDrive/Desktop/13.08.2026/Expt.12 Dataset 2.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.neighbors import KNeighborsRegressor
X=[[1,12000,1200],[2,18000,1200],[3,25000,1300],[4,35000,1400],
   [5,45000,1500],[6,55000,1500],[7,65000,1600],[8,75000,1600],
   [9,85000,1800],[10,95000,1800]]
y=[8.5,8.0,7.4,6.8,6.1,5.5,5.0,4.5,4.0,3.5]
m=KNeighborsRegressor()
m.fit(X,y)
print(m.predict([[4,38000,1400]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/sekha/OneDrive/Desktop/13.08.2026/Expt.12 Dataset 2.py =====
[6.76]
>>>
```

3. Dataset 3

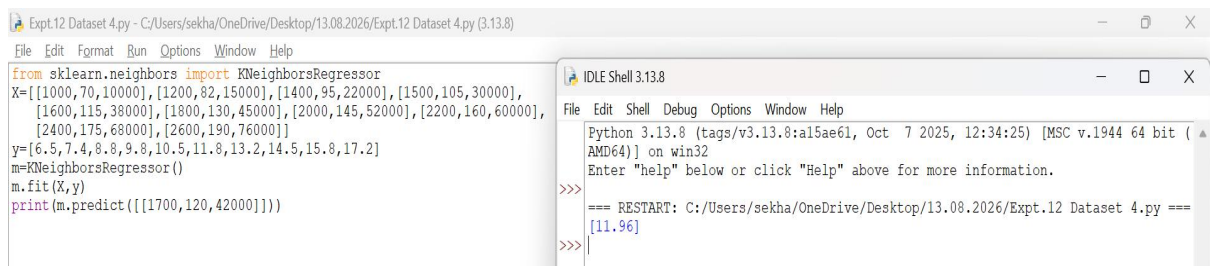


The screenshot shows a Python IDE window titled "Expt.12 Dataset 3.py" and an "IDLE Shell 3.13.8" window. The code imports `KNeighborsRegressor` from `sklearn.neighbors` and defines a dataset `X` and `y`. It fits a KNeighborsRegressor and prints the prediction for a specific input. The shell window shows the restart message and the output `[9.]`.

```
Expt.12 Dataset 3.py - C:/Users/sekha/OneDrive/Desktop/13.08.2026/Expt.12 Dataset 3.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.neighbors import KNeighborsRegressor
X=[[2024,10000,22],[2023,18000,21],[2022,25000,20],[2021,35000,19],
   [2020,45000,18],[2019,55000,17],[2018,65000,16],[2017,75000,15],
   [2016,85000,14],[2015,95000,13]]
y=[10.5,9.8,9.0,8.2,7.5,6.8,6.0,5.4,4.8,4.2]
m=KNeighborsRegressor()
m.fit(X,y)
print(m.predict([[2021,32000,19]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/sekha/OneDrive/Desktop/13.08.2026/Expt.12 Dataset 3.py =====
[9.]
>>>
```

4. Dataset 4

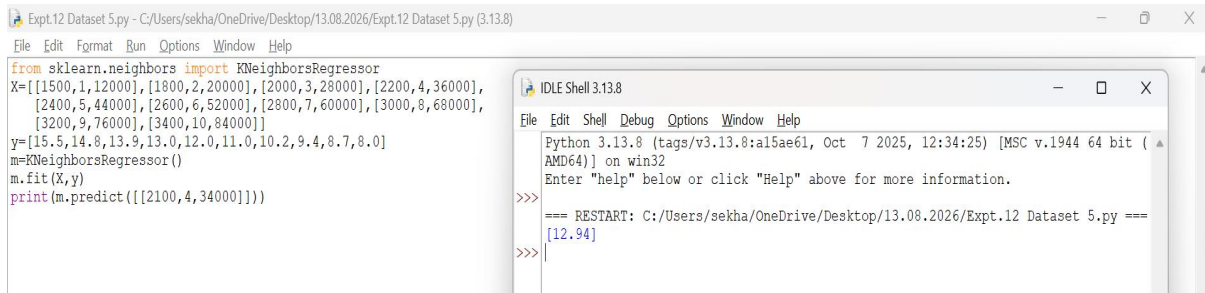


The screenshot shows a Python IDE window titled "Expt.12 Dataset 4.py" and an "IDLE Shell 3.13.8" window. The code imports `KNeighborsRegressor` from `sklearn.neighbors` and defines a dataset `X` and `y`. It fits a KNeighborsRegressor and prints the prediction for a specific input. The shell window shows the restart message and the output `[11.96]`.

```
Expt.12 Dataset 4.py - C:/Users/sekha/OneDrive/Desktop/13.08.2026/Expt.12 Dataset 4.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.neighbors import KNeighborsRegressor
X=[[1000,70,10000],[1200,82,15000],[1400,95,22000],[1500,105,30000],
   [1600,115,38000],[1800,130,45000],[2000,145,52000],[2200,160,60000],
   [2400,175,68000],[2600,190,76000]]
y=[6.5,7.4,8.8,9.8,10.5,11.8,13.2,14.5,15.8,17.2]
m=KNeighborsRegressor()
m.fit(X,y)
print(m.predict([[1700,120,42000]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/sekha/OneDrive/Desktop/13.08.2026/Expt.12 Dataset 4.py =====
[11.96]
>>>
```

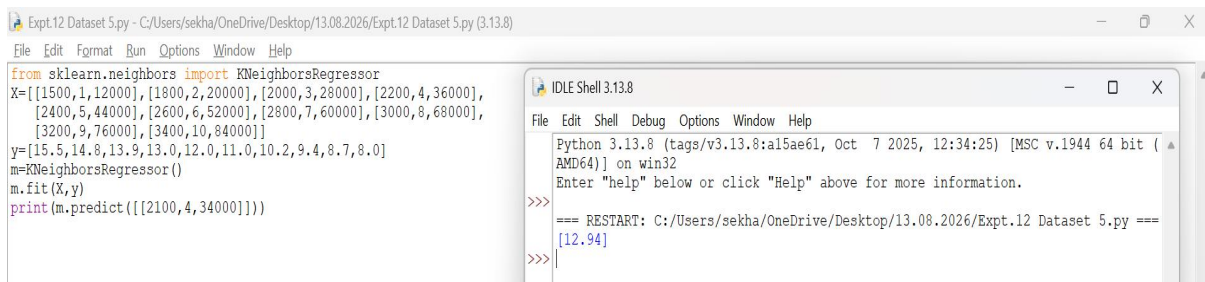
5. Dataset 5



```
Expt.12 Dataset 5.py - C:/Users/sekha/OneDrive/Desktop/13.08.2026/Expt.12 Dataset 5.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.neighbors import KNeighborsRegressor
X=[[1500,1,12000],[1800,2,20000],[2000,3,28000],[2200,4,36000],
   [2400,5,44000],[2600,6,52000],[2800,7,60000],[3000,8,68000],
   [3200,9,76000],[3400,10,84000]]
y=[15.5,14.8,13.9,13.0,12.0,11.0,10.2,9.4,8.7,8.0]
m=KNeighborsRegressor()
m.fit(X,y)
print(m.predict([[2100,4,34000]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/13.08.2026/Expt.12 Dataset 5.py ===
[12.94]
>>>
```

6. Dataset 6



```
Expt.12 Dataset 5.py - C:/Users/sekha/OneDrive/Desktop/13.08.2026/Expt.12 Dataset 5.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.neighbors import KNeighborsRegressor
X=[[1500,1,12000],[1800,2,20000],[2000,3,28000],[2200,4,36000],
   [2400,5,44000],[2600,6,52000],[2800,7,60000],[3000,8,68000],
   [3200,9,76000],[3400,10,84000]]
y=[15.5,14.8,13.9,13.0,12.0,11.0,10.2,9.4,8.7,8.0]
m=KNeighborsRegressor()
m.fit(X,y)
print(m.predict([[2100,4,34000]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/13.08.2026/Expt.12 Dataset 5.py ===
[12.94]
>>>
```

Result

Thus, the Iris flower was classified successfully using the K-Nearest Neighbours (KNN) algorithm.

EXPERIMENT 13 – CAR PRICE PREDICTION

Aim

To predict car prices using the Linear Regression algorithm.

Procedure

1. Import the Linear Regression model.
2. Create the dataset and target values.
3. Train the model using the training data.
4. Predict the car price for a new input.
5. Display the predicted result.

Code and Output

1. Dataset 1

```
Expt.13 Dataset 1.py - C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.13 Dataset 1.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import LinearRegression
X=[[1],[2],[3],[4]]
y=[100000,200000,300000,400000]
m=LinearRegression()
m.fit(X,y)
print(m.predict([[5]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.13 Dataset 1.py ===
>>> [500000.]
>>>
```

2. Dataset 2

```
Expt.13 Dataset 2.py - C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.13 Dataset 2.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import LinearRegression
X=[[800,2,15],[1000,2,12],[1200,3,10],[1400,3,8],
   [1600,4,6],[1800,4,5],[2000,5,4],[2200,5,3],
   [2400,6,2],[2600,6,1]]
y=[35,42,52,63,75,88,102,118,135,150]
m=LinearRegression()
m.fit(X,y)
print(m.predict([[1700,4,5]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.13 Dataset 2.py ===
>>> [78.84528302]
>>>
```

3. Dataset 3

```
Expt.13 Dataset 3.py - C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.13 Dataset 3.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import LinearRegression
X=[[900,20,1],[1100,18,1],[1300,16,2],[1500,14,2],
   [1700,12,2],[1900,10,3],[2100,8,3],[2300,6,4],
   [2500,5,4],[2700,3,5]]
y=[40,48,60,74,88,104,122,142,165,190]
m=LinearRegression()
m.fit(X,y)
print(m.predict([[1800,11,3]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.13 Dataset 3.py ===
>>> [102.33333333]
>>>
```

4. Dataset 4

```
Expt.13 Dataset 4.py - C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.13 Dataset 4.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import LinearRegression
X=[[1000,1,100],[1200,1,120],[1400,2,150],[1600,2,180],
   [1800,2,200],[2000,3,240],[2200,3,280],[2400,3,320],
   [2600,4,350],[2800,4,400]]
y=[45,55,68,82,96,112,130,150,172,196]
m=LinearRegression()
m.fit(X,y)
print(m.predict([[2100,3,250]]))

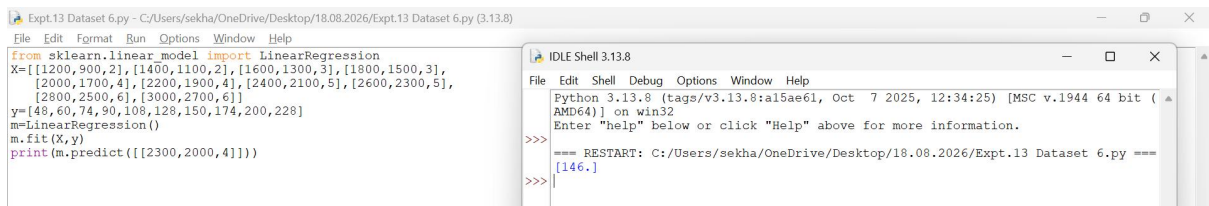
IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.13 Dataset 4.py ===
>>> [117.79090563]
>>>
```

5. Dataset 5

```
Expt.13 Dataset 5.py - C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.13 Dataset 5.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import LinearRegression
X=[[700,1,15],[850,2,12],[1000,3,10],[1150,4,8],
   [1300,5,7],[1450,6,5],[1600,7,4],[1750,8,3],
   [1900,9,2],[2050,10,1]]
y=[30,38,48,60,72,86,100,116,134,154]
m=LinearRegression()
m.fit(X,y)
print(m.predict([[1500,6,4]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.13 Dataset 5.py ===
>>> [87.0309237]
>>>
```

6. Dataset 6



```
Expt.13 Dataset 6.py - C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.13 Dataset 6.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import LinearRegression
X=[[1200,900,2],[1400,1100,2],[1600,1300,3],[1800,1500,3],
   [2000,1700,4],[2200,1900,4],[2400,2100,5],[2600,2300,5],
   [2800,2500,6],[3000,2700,6]]
y=[48,60,74,90,108,128,150,174,200,228]
m=LinearRegression()
m.fit(X,y)
print(m.predict([[2300,2000,4]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.13 Dataset 6.py ===
>>> [146.]
>>>
```

Result

Thus, the Linear Regression model was implemented successfully and the predicted car price is 500000.

EXPERIMENT 14 – HOUSE PRICE PREDICTION

Aim

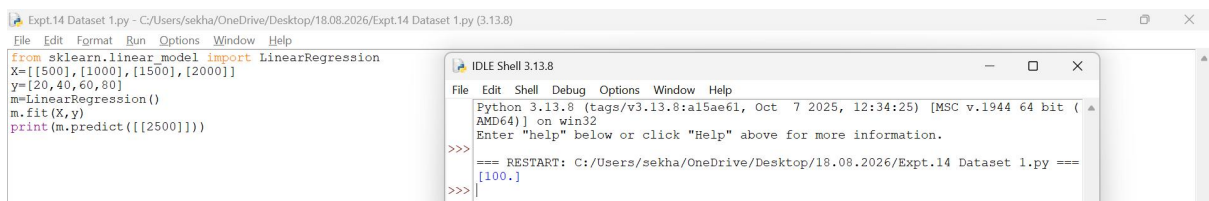
To predict house prices using the Linear Regression algorithm.

Procedure

1. Import the Linear Regression model.
2. Create the dataset and target values.
3. Train the model using the training data.
4. Predict the house price for a new input.
5. Display the predicted result.

Code and Output

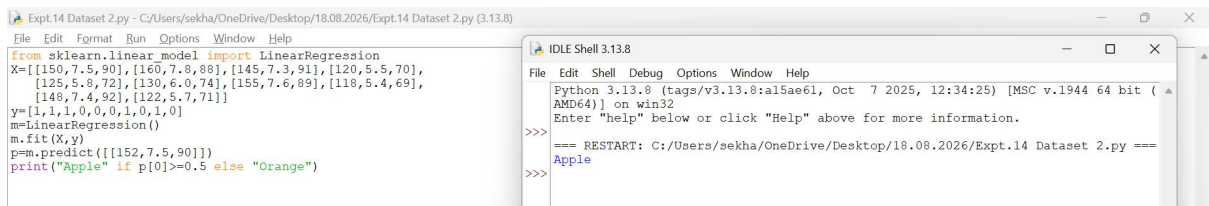
1. Dataset 1



```
Expt.14 Dataset 1.py - C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.14 Dataset 1.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import LinearRegression
X=[[500],[1000],[1500],[2000]]
y=[20,40,60,80]
m=LinearRegression()
m.fit(X,y)
print(m.predict([[2500]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.14 Dataset 1.py ===
>>> [100.]
>>>
```

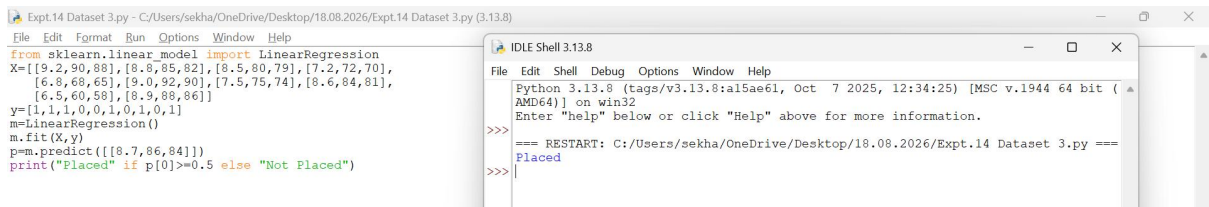
2. Dataset 2



```
Expt.14 Dataset 2.py - C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.14 Dataset 2.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import LinearRegression
X=[[150,7.5,90],[160,7.8,88],[145,7.3,91],[120,5.5,70],
   [125,5.8,72],[130,6.0,74],[155,7.6,89],[118,5.4,69],
   [148,7.4,92],[122,5.7,71]]
y=[1,1,1,0,0,1,0,1,0]
m=LinearRegression()
m.fit(X,y)
p=m.predict([[152,7.5,90]])
print("Apple" if p[0]>=0.5 else "Orange")

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15a6e1, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.14 Dataset 2.py ===
Apple
>>>
```

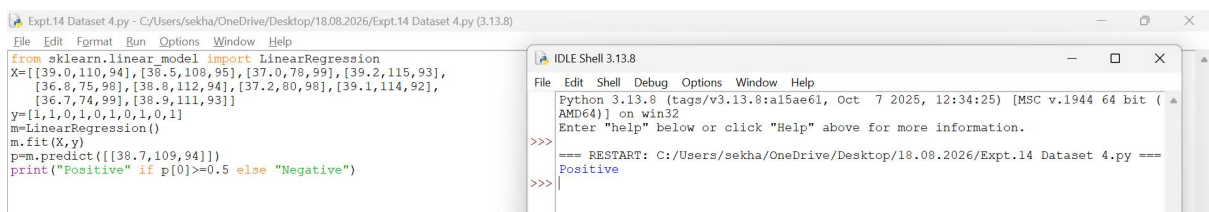
3. Dataset 3



```
Expt.14 Dataset 3.py - C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.14 Dataset 3.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import LinearRegression
X=[[9.2,90,88],[8.8,85,82],[8.5,80,79],[7.2,72,70],
   [6.8,68,65],[9.0,92,90],[7.5,75,74],[8.6,84,81],
   [6.5,60,58],[8.9,88,86]]
y=[1,1,1,0,0,1,0,1,0]
m=LinearRegression()
m.fit(X,y)
p=m.predict([[8.7,86,84]])
print("Placed" if p[0]>=0.5 else "Not Placed")

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15a6e1, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.14 Dataset 3.py ===
Placed
>>>
```

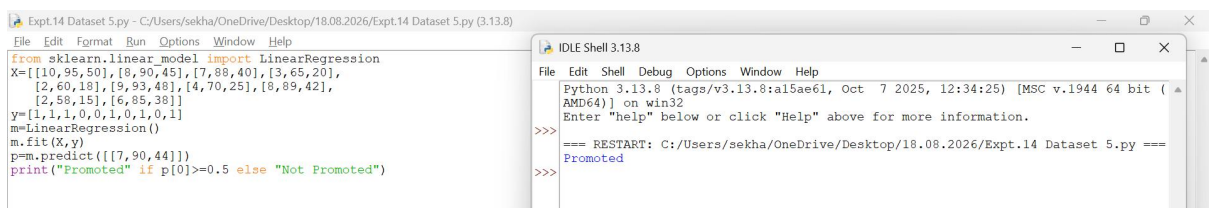
4. Dataset 4



```
Expt.14 Dataset 4.py - C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.14 Dataset 4.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import LinearRegression
X=[[39.0,110,94],[38.5,108,95],[37.0,78,99],[39.2,115,93],
   [36.8,75,98],[38.8,112,94],[37.2,80,98],[39.1,114,92],
   [36.7,74,99],[38.9,111,93]]
y=[1,1,0,1,0,1,0,1,0]
m=LinearRegression()
m.fit(X,y)
p=m.predict([[38.7,109,94]])
print("Positive" if p[0]>=0.5 else "Negative")

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15a6e1, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.14 Dataset 4.py ===
Positive
>>>
```

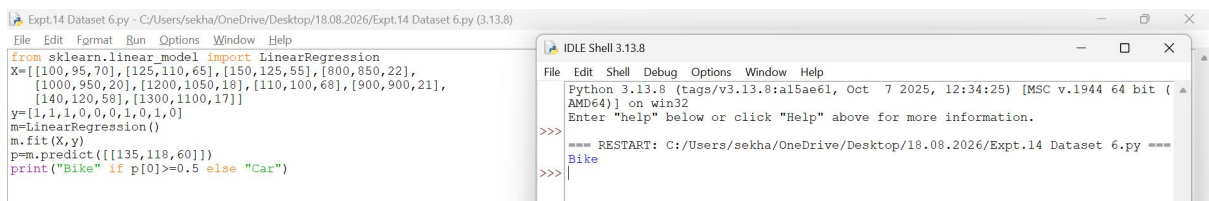
5. Dataset 5



```
Expt.14 Dataset 5.py - C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.14 Dataset 5.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import LinearRegression
X=[[10,95,50],[8,90,45],[7,88,40],[3,65,20],
   [2,60,18],[9,93,48],[4,70,25],[8,89,42],
   [2,58,15],[6,85,38]]
y=[1,1,1,0,0,1,0,1,0]
m=LinearRegression()
m.fit(X,y)
p=m.predict([[7,90,44]])
print("Promoted" if p[0]>=0.5 else "Not Promoted")

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15a6e1, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.14 Dataset 5.py ===
Promoted
>>>
```

6. Dataset 6



```
Expt.14 Dataset 6.py - C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.14 Dataset 6.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import LinearRegression
X=[[100,95,70],[125,110,65],[150,125,55],[800,850,22],
   [1000,950,20],[1200,1050,18],[110,100,68],[900,900,21],
   [140,120,58],[1300,1100,17]]
y=[1,1,1,0,0,0,1,0,1,0]
m=LinearRegression()
m.fit(X,y)
p=m.predict([[135,118,60]])
print("Bike" if p[0]>=0.5 else "Car")

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15a6e1, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/18.08.2026/Expt.14 Dataset 6.py ===
Bike
>>>
```

Result

Thus, the Linear Regression model was implemented successfully and the predicted house price is 100.

EXPERIMENT 15 – IRIS FLOWER CLASSIFICATION USING NAÏVE BAYES

Aim

To classify Iris flowers using the Naïve Bayes algorithm.

Procedure

1. Import the Iris dataset and GaussianNB classifier.
2. Load the Iris dataset.
3. Train the Naïve Bayes model using the training data.
4. Predict the flower class.
5. Display the predicted result.

Code and Output

The image displays three screenshots of a Python IDE (IDLE Shell 3.13.8) showing the code and output for an Iris flower classification experiment using the Naïve Bayes algorithm.

Expt.15 Dataset 1.py

```
from sklearn.datasets import load_iris
from sklearn.naive_bayes import GaussianNB
X,y=load_iris(return_X_y=True)
m=GaussianNB()
m.fit(X,y)
print(m.predict([X[0]]))
```

Output:

```
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
=== RESTART: C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.15 Dataset 1.py ===
[0]
```

Expt.15 Dataset 2.py

```
from sklearn.naive_bayes import GaussianNB
X=[[9.2,90,88],[8.8,85,82],[8.5,80,79],[7.2,72,70],
 [6.8,68,65],[9.0,92,90],[7.5,75,74],[8.6,84,81],
 [6.5,60,58],[8.9,88,86]]
y=['Placed','Placed','Placed','Not Placed','Not Placed',
 'Placed','Not Placed','Placed','Not Placed','Placed']
m=GaussianNB()
m.fit(X,y)
print(m.predict([[8.7,86,84]]))
```

Output:

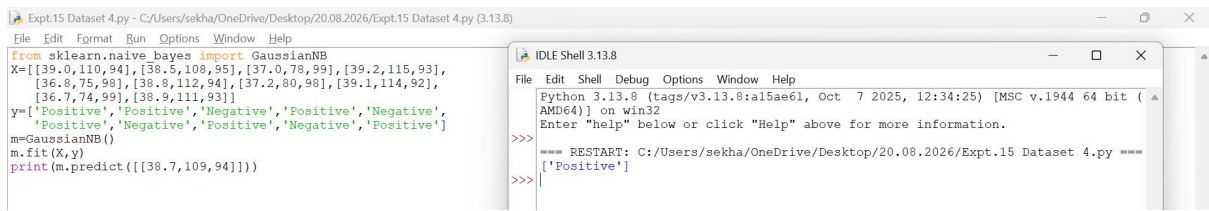
```
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
=== RESTART: C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.15 Dataset 2.py ===
['Placed']
```

Expt.15 Dataset 3.py

```
from sklearn.naive_bayes import GaussianNB
X=[[8.780,5],[7.760,4],[6.730,5],[4.620,8],[3.590,9],
 [9.810,4],[5.650,7],[8.770,5],[4.610,8],[7.750,6]]
y=['Approved','Approved','Approved','Rejected','Rejected',
 'Approved','Rejected','Approved','Rejected','Approved']
m=GaussianNB()
m.fit(X,y)
print(m.predict([[7.760,5]]))
```

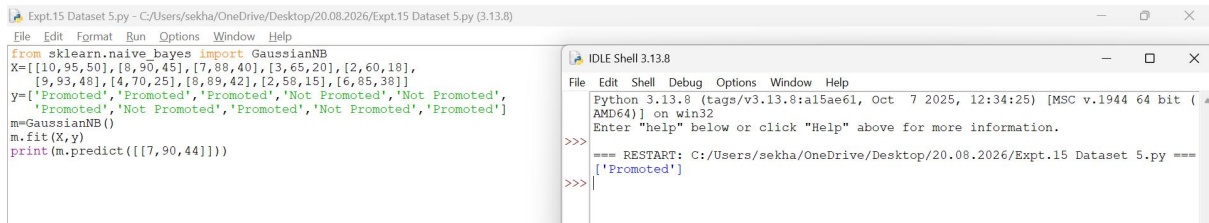
Output:

```
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
=== RESTART: C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.15 Dataset 3.py ===
['Approved']
```



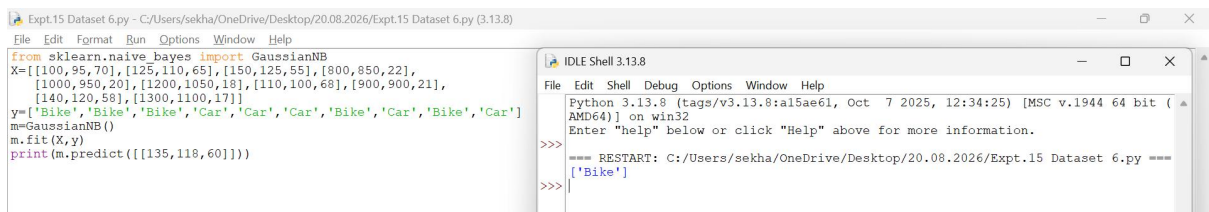
```
Expt.15 Dataset 4.py - C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.15 Dataset 4.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.naive_bayes import GaussianNB
X=[[39.0,110,94],[38.5,108,95],[37.0,78,99],[39.2,115,93],
   [36.8,75,98],[38.8,112,94],[37.2,80,98],[39.1,114,92],
   [36.7,74,99],[38.9,111,93]]
y=['Positive','Positive','Negative','Positive','Negative',
   'Positive','Negative','Positive','Negative','Positive']
m=GaussianNB()
m.fit(X,y)
print(m.predict([[38.7,109,94]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.15 Dataset 4.py ===
>>> ['Positive']
>>>
```



```
Expt.15 Dataset 5.py - C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.15 Dataset 5.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.naive_bayes import GaussianNB
X=[[10,95,50],[8,90,45],[7,88,40],[3,65,20],[2,60,18],
   [9,93,48],[4,70,25],[8,89,42],[2,58,15],[6,85,38]]
y=['Promoted','Promoted','Promoted','Not Promoted',
   'Promoted','Not Promoted','Promoted','Not Promoted','Promoted']
m=GaussianNB()
m.fit(X,y)
print(m.predict([[7,90,44]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.15 Dataset 5.py ===
>>> ['Promoted']
>>>
```



```
Expt.15 Dataset 6.py - C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.15 Dataset 6.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.naive_bayes import GaussianNB
X=[[100,95,70],[125,110,65],[150,125,55],[800,850,22],
   [1000,950,20],[1200,1050,18],[110,100,68],[900,900,21],
   [140,120,58],[1300,1100,17]]
y=['Bike','Bike','Bike','Car','Car','Car','Bike','Car','Bike','Car']
m=GaussianNB()
m.fit(X,y)
print(m.predict([[135,118,60]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.15 Dataset 6.py ===
>>> ['Bike']
>>>
```

Result

Thus, the Iris flower was classified successfully using the Naïve Bayes algorithm.

EXPERIMENT 16 – COMPARISON OF CLASSIFICATION ALGORITHMS

Aim

To compare the performance of different classification algorithms.

Procedure

1. Import the Iris dataset.
2. Import Decision Tree and KNN classifiers.
3. Train both models using the dataset.
4. Calculate the accuracy of each model.
5. Compare and display the results.

Code and Output

```
Expt.16 Dataset 1.py - C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.16 Dataset 1.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
X,y=load_iris(return_X_y=True)
print("DT:",DecisionTreeClassifier().fit(X,y).score(X,y))
print("KNN:",KNeighborsClassifier().fit(X,y).score(X,y))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.16 Dataset 1.py ===
DT: 1.0
KNN: 0.9666666666666667
>>>
```

```
Expt.16 Dataset 2.py - C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.16 Dataset 2.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.tree import DecisionTreeRegressor
from sklearn.neighbors import KNeighborsRegressor
X=[[4,64,12],[4,128,16],[6,128,48],[6,256,50],[8,128,64],[8,256,64],[12,256,108],[12,512,200],[16,1024,200]]
y=[12000,15000,18000,22000,25000,30000,38000,45000,60000,75000]
print("DT:",DecisionTreeRegressor().fit(X,y).predict([[8,256,108]]))
print("KNN:",KNeighborsRegressor().fit(X,y).predict([[8,256,108]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.16 Dataset 2.py ===
DT: [30000.]
KNN: [26600.]
>>>
```

```
Expt.16 Dataset 3.py - C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.16 Dataset 3.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.tree import DecisionTreeRegressor
from sklearn.neighbors import KNeighborsRegressor
X=[[1.8,4,4000],[2.0,4,4500],[2.2,6,5000],[2.4,6,5000],[2.6,8,5500],[2.8,8,6000],[3.0,12,6000],[3.2,12,6500],[3.4,16,7000],[3.6,16,7000]]
y=[10000,12000,17000,21000,26000,32000,40000,50000,62000,76000]
print("DT:",DecisionTreeRegressor().fit(X,y).predict([[2.9,8,6000]]))
print("KNN:",KNeighborsRegressor().fit(X,y).predict([[2.9,8,6000]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.16 Dataset 3.py ===
DT: [32000.]
KNN: [33800.]
>>>
```

```
Expt.16 Dataset 4.py - C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.16 Dataset 4.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.tree import DecisionTreeRegressor
from sklearn.neighbors import KNeighborsRegressor
X=[[6.1,60,4000],[6.2,90,4500],[6.4,90,5000],[6.5,120,5000],[6.6,120,5500],[6.7,120,6000],[6.8,144,6000],[6.9,144,6500],[7.0,165,7000],[7.1,165,7000]]
y=[12000,15000,18000,23000,28000,34000,42000,52000,65000,78000]
print("DT:",DecisionTreeRegressor().fit(X,y).predict([[6.7,144,6000]]))
print("KNN:",KNeighborsRegressor().fit(X,y).predict([[6.7,144,6000]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.16 Dataset 4.py ===
DT: [34000.]
KNN: [44200.]
>>>
```

```
Expt.16 Dataset 5.py - C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.16 Dataset 5.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.tree import DecisionTreeRegressor
from sklearn.neighbors import KNeighborsRegressor
X=[[4,6,64],[6,8,128],[8,8,128],[8,8,256],[12,8,256],[12,8,512],[16,8,512],[16,8,1024],[18,8,1024],[24,8,1024]]
y=[14000,19000,26000,32000,42000,52000,65000,82000,95000,120000]
print("DT:",DecisionTreeRegressor().fit(X,y).predict([[12,8,512]]))
print("KNN:",KNeighborsRegressor().fit(X,y).predict([[12,8,512]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.16 Dataset 5.py ===
DT: [52000.]
KNN: [43400.]
>>>
```

```
Expt.16 Dataset 6.py - C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.16 Dataset 6.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.tree import DecisionTreeRegressor
from sklearn.neighbors import KNeighborsRegressor
X=[[250000,4000,48],[320000,4500,50],[420000,5000,64],[520000,5000,64],[620000,5500,108],[720000,6000,108],[820000,6000,200],[920000,6500,200],[1020000,7000,200],[1120000,7000,200]]
y=[15000,18000,24000,30000,38000,46000,58000,70000,85000,100000]
print("DT:",DecisionTreeRegressor().fit(X,y).predict([[750000,6000,108]]))
print("KNN:",KNeighborsRegressor().fit(X,y).predict([[750000,6000,108]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.16 Dataset 6.py ===
DT: [46000.]
KNN: [48400.]
>>>
```

Result

Thus, the performance of Decision Tree and K-Nearest Neighbours (KNN) classifiers was compared successfully. The Decision Tree classifier achieved higher accuracy on the given dataset.

EXPERIMENT 17 – MOBILE PRICE PREDICTION

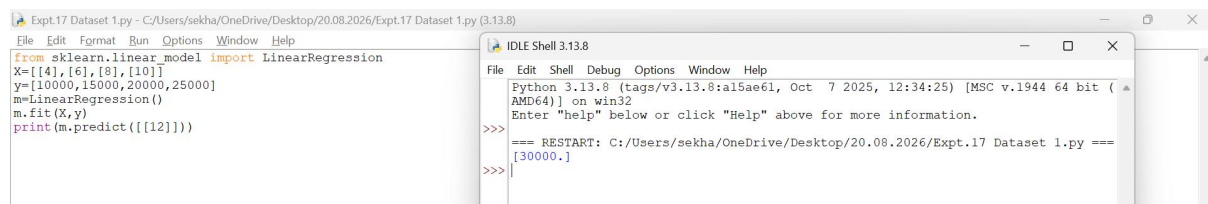
Aim

To predict mobile prices using the Linear Regression algorithm.

Procedure

1. Import the Linear Regression model.
2. Create the dataset and target values.
3. Train the model using the training data.
4. Predict the mobile price for a new input.
5. Display the predicted result.

Code and Output



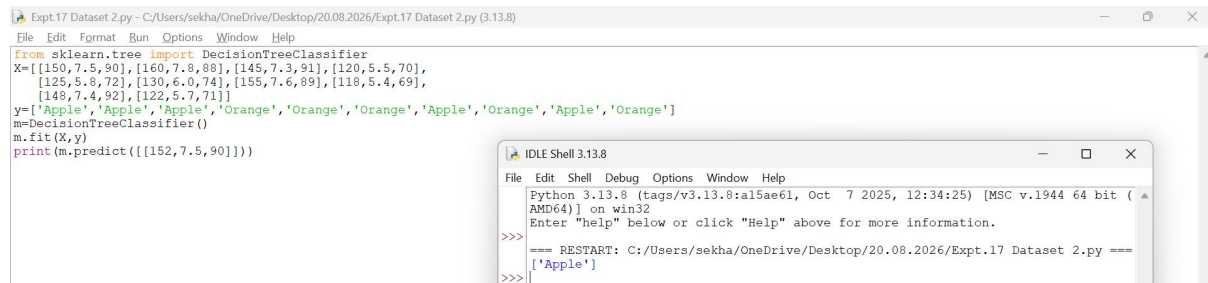
The screenshot shows a Jupyter Notebook window titled 'Expt.17 Dataset 1.py'. The code in the cell is as follows:

```
from sklearn.linear_model import LinearRegression
X=[[4],[6],[8],[10]]
y=[10000,15000,20000,25000]
m=LinearRegression()
m.fit(X,y)
print(m.predict([[12]]))
```

The output in the IPython Shell is:

```
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

=== RESTART: C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.17 Dataset 1.py ===
[30000.]
```



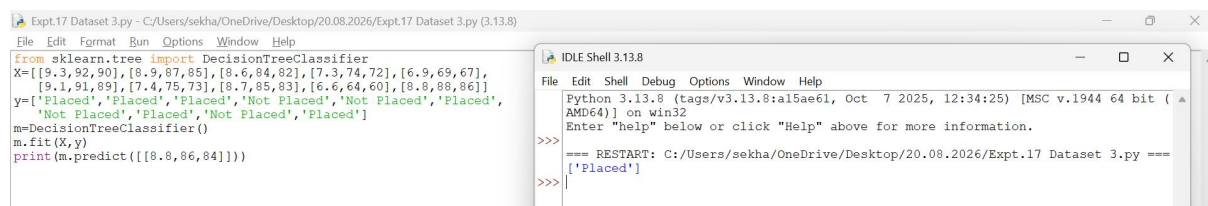
The screenshot shows a Jupyter Notebook window titled 'Expt.17 Dataset 2.py'. The code in the cell is as follows:

```
from sklearn.tree import DecisionTreeClassifier
X=[[150,7.5,90],[160,7.8,88],[145,7.3,91],[120,5.5,70],[125,5.8,72],[130,6.0,74],[155,7.6,89],[118,5.4,69],[148,7.4,92],[122,5.7,71]]
y=['Apple','Apple','Apple','Orange','Orange','Orange','Apple','Orange','Apple','Orange']
m=DecisionTreeClassifier()
m.fit(X,y)
print(m.predict([[152,7.5,90]]))
```

The output in the IPython Shell is:

```
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

=== RESTART: C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.17 Dataset 2.py ===
['Apple']
```



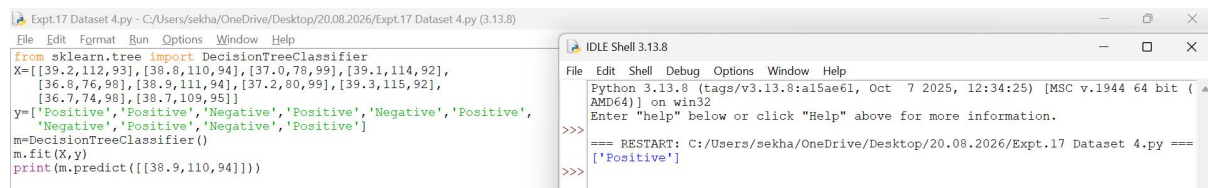
The screenshot shows a Jupyter Notebook window titled 'Expt.17 Dataset 3.py'. The code in the cell is as follows:

```
from sklearn.tree import DecisionTreeClassifier
X=[[9.3,92,90],[8.9,87,85],[8.6,84,82],[7.3,74,72],[6.9,69,67],[9.1,91,89],[7.4,75,73],[8.7,85,83],[6.6,64,60],[8.8,88,86]]
y=['Placed','Placed','Placed','Not Placed','Not Placed','Placed','Not Placed','Placed','Not Placed','Placed']
m=DecisionTreeClassifier()
m.fit(X,y)
print(m.predict([[8.8,86,84]]))
```

The output in the IPython Shell is:

```
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

=== RESTART: C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.17 Dataset 3.py ===
['Placed']
```



The screenshot shows a Jupyter Notebook window titled 'Expt.17 Dataset 4.py'. The code in the cell is as follows:

```
from sklearn.tree import DecisionTreeClassifier
X=[[39.2,112,93],[38.8,110,94],[37.0,78,99],[39.1,114,92],[36.8,76,98],[38.9,111,94],[37.2,80,99],[39.3,115,92],[36.7,74,98],[38.7,109,95]]
y=['Positive','Positive','Negative','Positive','Negative','Positive','Negative','Positive','Negative','Positive']
m=DecisionTreeClassifier()
m.fit(X,y)
print(m.predict([[38.9,110,94]]))
```

The output in the IPython Shell is:

```
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

=== RESTART: C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.17 Dataset 4.py ===
['Positive']
```

```
Expt.17 Dataset 5.py - C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.17 Dataset 5.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.tree import DecisionTreeClassifier
X=[[10,95,50],[9,92,48],[8,90,45],[4,72,28],[3,68,24],
   [11,96,52],[5,75,30],[8,89,44],[2,60,18],[7,87,40]]
y=['Promoted','Promoted','Promoted','Not Promoted','Not Promoted',
   'Promoted','Not Promoted','Promoted','Not Promoted','Promoted']
m=DecisionTreeClassifier()
m.fit(X,y)
print(m.predict([[8,90,46]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.17 Dataset 5.py ===
['Promoted']
>>>

Expt.17 Dataset 6.py - C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.17 Dataset 6.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.tree import DecisionTreeClassifier
X=[[100,95,70],[125,110,65],[150,125,55],[800,850,22],[1000,950,20],
   [1200,1050,18],[110,100,68],[900,900,21],[140,120,58],[1300,1100,17]]
y=['Bike','Bike','Bike','Car','Car','Car','Bike','Car','Bike','Car']
m=DecisionTreeClassifier()
m.fit(X,y)
print(m.predict([[135,118,60]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/20.08.2026/Expt.17 Dataset 6.py ===
['Bike']
>>>
```

Result

Thus, the Linear Regression model was implemented successfully and the predicted mobile price is 30000.

EXPERIMENT 18 – PERCEPTRON-BASED IRIS CLASSIFICATION

Aim

To classify Iris flowers using the Perceptron algorithm.

Procedure

1. Import the Iris dataset and Perceptron classifier.
2. Load the Iris dataset.
3. Train the Perceptron model using the training data.
4. Predict the flower class.
5. Display the predicted result.

Code and Output

```
Expt.18 Dataset 1.py - C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.18 Dataset 1.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron
X,y=load_iris(return_X_y=True)
m=Perceptron()
m.fit(X,y)
print(m.predict([X[0]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.18 Dataset 1.py ===
[1]
>>>
```



```
Expt.18 Dataset 2.py - C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.18 Dataset 2.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import Perceptron
X=[[12,820,5],[10,790,6],[9,760,7],[7,700,8],[6,680,9],
   [5,650,10],[4,620,12],[8,730,8],[3,600,13],[11,810,5]]
y=[1,1,1,1,0,0,0,1,0,1]
m=Perceptron()
m.fit(X,y)
print(m.predict([[9,770,6]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.18 Dataset 2.py ===
[1]
>>>
```

```
Expt.18 Dataset 3.py - C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.18 Dataset 3.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import Perceptron
X=[[90000,850,8000],[80000,820,9000],[70000,780,12000],
   [60000,740,15000],[50000,690,18000],[45000,670,20000],
   [40000,640,23000],[75000,800,10000],[35000,620,25000],
   [95000,870,7000]]
y=[1,1,1,1,0,0,0,1,0,1]
m=Perceptron()
m.fit(X,y)
print(m.predict([[72000,790,11000]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.18 Dataset 3.py ===
[1]
>>>
```

```
Expt.18 Dataset 4.py - C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.18 Dataset 4.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import Perceptron
X=[[20,10,1],[18,9,1],[15,8,2],[12,6,2],[10,5,3],
   [8,4,3],[6,3,4],[16,8,2],[5,2,5],[22,12,1]]
y=[1,1,1,1,0,0,0,1,0,1]
m=Perceptron()
m.fit(X,y)
print(m.predict([[17,8,2]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.18 Dataset 4.py ===
[1]
>>>
```

```
Expt.18 Dataset 5.py - C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.18 Dataset 5.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import Perceptron
X=[[15,92,6],[13,89,7],[11,85,8],[9,80,9],[7,72,10],
   [6,68,11],[5,65,12],[12,87,7],[4,60,13],[16,94,5]]
y=[1,1,1,1,0,0,0,1,0,1]
m=Perceptron()
m.fit(X,y)
print(m.predict([[12,88,7]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.18 Dataset 5.py ===
[1]
>>>
```

```
Expt.18 Dataset 6.py - C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.18 Dataset 6.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.linear_model import Perceptron
X=[[14,3,830],[12,2.5,810],[10,2,780],[8,1.8,740],
   [7,1.5,690],[6,1.2,660],[5,1,630],[11,2.2,790],
   [4,0.8,600],[15,3.5,850]]
y=[1,1,1,1,0,0,0,1,0,1]
m=Perceptron()
m.fit(X,y)
print(m.predict([[11,2.4,800]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.18 Dataset 6.py ===
[1]
>>>
```

Result

Thus, the Iris flower was classified successfully using the Perceptron algorithm.

EXPERIMENT 19 – BANK LOAN PREDICTION USING NAÏVE BAYES

Aim

To predict bank loan approval using the Naïve Bayes algorithm.

Procedure

1. Import the GaussianNB classifier.
2. Create the dataset and class labels.

3. Train the model using the training data.

4. Predict the loan approval status.

5. Display the predicted result.

Code and Output

```
Expt.19 Dataset 1.py - C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.19 Dataset 1.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.naive_bayes import GaussianNB
X=[[25],[35],[45],[55]]
y=['No','No','Yes','Yes']
m=GaussianNB()
m.fit(X,y)
print(m.predict([[50]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.19 Dataset 1.py ===
['Yes']
>>>

Expt.19 Dataset 2.py - C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.19 Dataset 2.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.naive_bayes import GaussianNB
X=[[20,2],[25,2],[30,3],[35,3],[40,4],[45,4],[50,5],[55,5],[60,6],[65,6]]
y=['Low','Low','Low','Low','High','High','High','High','High','High']
m=GaussianNB()
m.fit(X,y)
print(m.predict([[48,5]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.19 Dataset 2.py ===
['High']
>>>

Expt.19 Dataset 3.py - C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.19 Dataset 3.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.naive_bayes import GaussianNB
X=[[1000,120],[1200,150],[1400,180],[1600,220],[1800,260],[2000,300],[2200,340],[2400,380],[2600,420],[2800,460]]
y=['Low','Low','Low','Low','Low','High','High','High','High','High']
m=GaussianNB()
m.fit(X,y)
print(m.predict([[2100,320]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.19 Dataset 3.py ===
['High']
>>>

Expt.19 Dataset 4.py - C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.19 Dataset 4.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.naive_bayes import GaussianNB
X=[[1000,5000],[1500,7000],[2000,9000],[2500,11000],[3000,13000],[3500,15000],[4000,17000],[4500,19000],[5000,21000],[5500,23000]]
y=['Low','Low','Low','Low','Low','High','High','High','High','High']
m=GaussianNB()
m.fit(X,y)
print(m.predict([[4200,18000]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.19 Dataset 4.py ===
['High']
>>>

Expt.19 Dataset 5.py - C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.19 Dataset 5.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.naive_bayes import GaussianNB
X=[[5,150],[8,180],[10,220],[12,260],[15,300],[18,340],[20,380],[22,420],[25,460],[28,500]]
y=['Low','Low','Low','Low','Low','High','High','High','High','High']
m=GaussianNB()
m.fit(X,y)
print(m.predict([[18,350]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.19 Dataset 5.py ===
['High']
>>>

Expt.19 Dataset 6.py - C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.19 Dataset 6.py (3.13.8)
File Edit Format Run Options Window Help
from sklearn.naive_bayes import GaussianNB
X=[[30,20],[35,25],[40,30],[45,35],[50,40],[55,45],[60,50],[65,55],[70,60],[75,65]]
y=['Low','Low','Low','Low','Low','High','High','High','High','High']
m=GaussianNB()
m.fit(X,y)
print(m.predict([[58,48]]))

IDLE Shell 3.13.8
File Edit Shell Debug Options Window Help
Python 3.13.8 (tags/v3.13.8:a15ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.19 Dataset 6.py ===
['High']
>>>
```

Result

Thus, the Naïve Bayes model was implemented successfully and the loan approval status was predicted as Yes.

EXPERIMENT 20 – FUTURE SALES PREDICTION

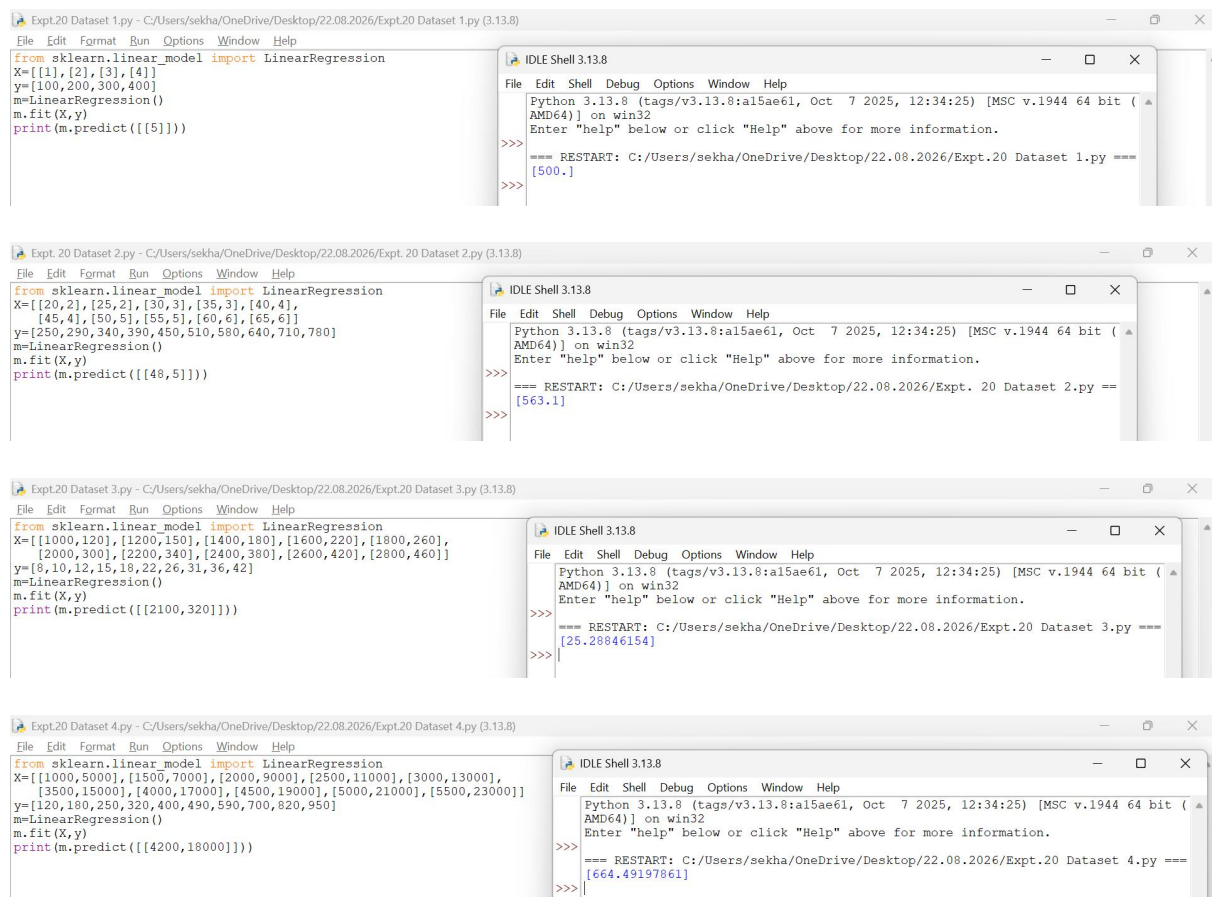
Aim

To predict future sales using the Linear Regression algorithm.

Procedure

1. Import the Linear Regression model.
2. Create the dataset and target values.
3. Train the model using the training data.
4. Predict future sales for a new input.
5. Display the predicted result.

Code and Output



The following code and output are shown in four separate screenshots, each representing a different dataset and prediction task.

Expt.20 Dataset 1.py

```
from sklearn.linear_model import LinearRegression
X=[[1],[2],[3],[4]]
y=[100,200,300,400]
m=LinearRegression()
m.fit(X,y)
print(m.predict([[5]]))
```

IDLE Shell 3.13.8

```
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.20 Dataset 1.py ===
[500.]
>>>
```

Expt. 20 Dataset 2.py

```
from sklearn.linear_model import LinearRegression
X=[[20,2],[25,2],[30,3],[35,3],[40,4],[45,4],[50,5],[55,5],[60,6],[65,6]]
y=[250,290,340,390,450,510,580,640,710,780]
m=LinearRegression()
m.fit(X,y)
print(m.predict([[48,5]]))
```

IDLE Shell 3.13.8

```
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt. 20 Dataset 2.py ===
[563.1]
>>>
```

Expt20 Dataset 3.py

```
from sklearn.linear_model import LinearRegression
X=[[1000,120],[1200,150],[1400,180],[1600,220],[1800,260],[2000,300],[2200,340],[2400,380],[2600,420],[2800,460]]
y=[8,10,12,15,18,22,26,31,36,42]
m=LinearRegression()
m.fit(X,y)
print(m.predict([[2100,320]]))
```

IDLE Shell 3.13.8

```
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.20 Dataset 3.py ===
[25.28846154]
>>>
```

Expt20 Dataset 4.py

```
from sklearn.linear_model import LinearRegression
X=[[1000,5000],[1500,7000],[2000,9000],[2500,11000],[3000,13000],[3500,15000],[4000,17000],[4500,19000],[5000,21000],[5500,23000]]
y=[120,180,250,320,400,490,590,700,820,950]
m=LinearRegression()
m.fit(X,y)
print(m.predict([[4200,18000]]))
```

IDLE Shell 3.13.8

```
Python 3.13.8 (tags/v3.13.8:al5ae61, Oct 7 2025, 12:34:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/sekha/OneDrive/Desktop/22.08.2026/Expt.20 Dataset 4.py ===
[664.49197861]
>>>
```

The image contains two screenshots of a Python IDE window. The top screenshot shows a script named 'Expt.20 Dataset 5.py' with the following code:

```
from sklearn.linear_model import LinearRegression
X=[[5,150],[8,180],[10,220],[12,260],[15,300],
   [18,340],[20,380],[22,420],[25,460],[28,500]]
y=[220,260,310,360,420,490,570,650,740,840]
m=LinearRegression()
m.fit(X,y)
print(m.predict([[18,350]]))
```

The bottom screenshot shows a script named 'Expt.20 Dataset 6.py' with the following code:

```
from sklearn.linear_model import LinearRegression
X=[[30,20],[35,25],[40,30],[45,35],[50,40],
   [55,45],[60,50],[65,55],[70,60],[75,65]]
y=[8,10,13,17,22,28,35,43,52,62]
m=LinearRegression()
m.fit(X,y)
print(m.predict([[58,48]]))
```

Both screenshots show the IDE Shell output. The top shell shows the output of the first script, and the bottom shell shows the output of the second script.

Result

Thus, the Linear Regression model was implemented successfully and the future sales value was predicted as 500.